Detecting Contradictions Within and Across Documents

Paper ID: 53

ABSTRACT

This paper presents DocChecker, a system for detecting textual contradictions. Given a document collection $\mathcal{D}$, DocChecker identifies conflicting properties or relationships assigned to the same entity, both within and across documents. It adopts a graph-based paradigm: entities and their relations are extracted, represented as a property graph G, joined across documents, and enriched with domain knowledge. DocChecker introduces a class of rules that detect vertex- and edge-level contradictions in PTIME, unifying logical reasoning, ML predictions, and NLP tools. We present how graphs are extracted, joined, and enriched, and propose a parallel rule discovery algorithm with new generation, evaluation, and validation strategies tailored for sparse contradictions. Using real-life documents, we empirically verify that DocChecker detects intra- and inter-document contradictions with an average F1-score of 0.900, up to 0.935, and it scales to large collections $\mathcal{D}$ in parallel.

1 INTRODUCTION

A contradiction is a condition in which an entity is simultaneously assigned two or more properties or relationships that lead to a logical inconsistency, *i.e.,* they are logically or semantically incompatible at the same time, such that their joint validity violates domain constraints or undermines internal consistency [16, 79, 109].

Documents contain various contradictions. Examples include: (a) event/entity inconsistencies, *e.g.,* "All donors are between 20–45 years old" vs. "Lisa, who donates her kidney, is 70"; (b) conflicting opinions, *e.g.,* "The project a breakthrough" vs. "The project had no impact"; and (c) mutually exclusive relations/knowledge, *e.g.,* "Jane and Tom are a married couple" vs. "Jane is Tom's sister" (cf. [79]). Such contradictions may occur within a single document (*intra-document*) or across related documents (*inter-document*).

Contradictions in documents are common. An analysis of news samples from 2007, 2013 and 2019 found that about 21% of news stories contained contradictions, with national newspapers experiencing an even higher rate of 24% [19]. Moreover, journalist-generated contradictions rose from 9.1% in 2007 to 16.6% in 2019.

The prevalence of textual inconsistencies highlights the need for their detection, since such contradictions can undermine core tasks including information retrieval, named entity resolution (NER), retrieval-augmented generation (RAG), reading comprehension, medical text analysis, and LLM fine-tuning [109, 119, 125, 128].

Early work on contradiction detection [14, 20, 22, 56, 104, 130] typically frames the task as text classification [56, 62] or textual entailment [20, 90, 104, 130]. However, such NLI (Natural Language Inference)-based methods are rather shallow [59, 87, 132] and struggle with cross-document conflicts or fine-grained inconsistencies in long contexts [23, 79]. LLMs seem promising but remain unreliable: even state-of-the-art (SOTA) models achieve only 51.6% accuracy [79], and often amplify noise, reinforce misinformation, or hallucinate. When contradictions arise, LLMs tend to default to parametric knowledge [15] or favor majority entities backed by larger volumes [125], rather than reasoning over conflicting evidence.

Challenges. Detecting textual contradictions is challenging.

(1) Context dependence. Contradictions often hinge on context that shifts within or across documents [16, 79]. A statement may appear consistent in isolation but become contradictory when interpreted alongside related claims elsewhere. For instance, "the vaccine is safe for children" and "the vaccine has not been tested on minors" only conflict when linked through contextual inference.

(2) Linguistic variation. Contradictions may be masked by paraphrasing, negation, or figurative language [7, 105, 109]. The same claim can be phrased in multiple ways; syntactic differences ("X increases risk" vs. "risk is higher with X") or pragmatic nuances ("may cause" vs. "is known to cause") can obscure semantic opposition.

(3) Domain specificity. In specialized domains, *e.g.,* medicine, law, or science, contradictions depend on expert knowledge of terminology, causality and regulatory logic [127]. For example, "aspirin prevents blood clots" and "aspirin increases bleeding risk" are both true, but they reflect opposing medical implications of the same mechanism; one statement highlights the benefit of aspirin, the other the risk.

(4) Temporal dynamics. Information evolves over time, and statements that were once consistent may later contradict due to policy changes, scientific updates, or data revisions [79, 86, 108, 109]. For example, "masks are unnecessary" (early 2020) and "masks are required" (late 2020) differ not by error but by temporal context.

(5) Scalability. As documents and corpora grow longer, exhaustive pairwise comparison of statements becomes prohibitively expensive [16, 127]. Practical contradiction detection must therefore scale to large, evolving corpora without sacrificing accuracy.

These challenges raise several questions: (1) How can contradictions of different types be detected in a unified way, capturing contextual cues, linguistic variation, and temporal change? (2) How can external domain knowledge be incorporated to boost accuracy when dealing with specialized terminology and context? (3) How can systems efficiently and consistently detect contradictions both within a document and across multiple documents at scale?

DocChecker. To answer these questions, we develop DocChecker, a system for detecting diverse types of contradictions across a document collection $\mathcal{D}$, integrating contextual reasoning, domain knowledge, and scalable analysis. It offers the following features.

(1) Rules. Underlying DocChecker is a class of Rules for Identifying COntradictions (RICOs) over text and graph data. RICOs encode entities and relations as graph patterns, and integrate logical reasoning with ML/NLP techniques to detect contradictions. They (a) combine reasoning and prediction to catch context and temporal change, (b) link graph vertices to document spans and use NLP tools to resolve linguistic variation, and (c) infer conflicts over both entity attributes and relations, catching diverse textual inconsistencies.

(2) A graph-based framework. Given a document collection $\mathcal{D}$, DocChecker detects contradictions in five steps: (a) convert each document into a property graph and enrich its entities and edges

with domain knowledge (KG, if available); (b) join these graphs by aligning vertices that refer to the same entity, yielding a fused graph G; (c) learn RICOs from samples of G to capture domain and document patterns; (d) detect contradictions in G using a parallel rule-based algorithm; and (e) map detected conflicts back to text. This enables unified detection of both intra- and inter-document contradictions within the fused graph, while respecting domain terminology, causal logic, and contextual conventions.

(3) Scalable algorithms. We develop a suite of graph analytic algorithms underlying DocChecker, including (a) a graph join algorithm to fuse extracted graphs into a unified graph G and enrich it with domain knowledge from KG, (b) Learner, an algorithm for learning RICOs from (samples of) G, and (c) contradiction detection algorithms for identifying inconsistencies in G. These algorithms are designed to scale to long documents and large collections.

The algorithms improve upon prior work. Taking Learner as an example, it (a) adopts a learning-based rule generator instead of levelwise traversal; (b) restricts rule generation to the contradiction-relevant subset $\mathcal{D}'$ of $\mathcal{D}$ to narrow candidates; (c) replaces support with downstream performance metrics suited to sparse contradictions; (d) introduces two thresholds (contradiction coverage and accuracy improvement) to ensure most contradictions in $\mathcal{D}$ are captured; and (e) aligns entities and relations from $\mathcal{D}'$ with triples in G, using their attributes as sources for RICO predicate generation.

(4) Effectiveness. Using real-life documents, we empirically verify DocChecker's accuracy and scalability: (a) it achieves SOTA results on 14 baselines, including 93.1% F_1 on SNLI [14] and 87.8% accuracy on ContractNLI [70]; (b) it excels at long-range reasoning, sustaining 76.81% (intra-) and 86.96% (inter-document) accuracy, beating GPT-4o (51.89%/65.06%); (c) in document-level self-contradiction detection, DocChecker provides verifiable evidence (0.962 F_1), exceeding Claude-3.7 and GPT-5.2; and (d) it scales with document size and entity count in parallel, with 16-processor execution reducing runtime from 7410.11 s to 974.44 s, enabling large-scale deployment.

Organization. Section 2 reviews document contradictions and prior work. Section 3 introduces RICOs, and proves that contradiction detection with RICOs can be performed in polynomial time (PTIME). Section 4 presents the workflow of DocChecker. Section 5 provides graph conversion and join algorithms. Section 6 develops algorithm Learner for discovering RICOs. Section 7 reports our experimental findings. Section 8 summarizes the novelty of the work.

Detailed proofs, the parallel contradiction detection algorithm, and document marking procedures are deferred to [1].

2 BACKGROUND AND RELATED WORK

This section reviews background and previous work.

Documents. Assume a countably infinite set of symbols W for tokens. A document D is represented as a finite token sequence $\langle w_1, w_2, \ldots, w_{|D|} \rangle$, where each token $w_i \in W$ denotes a word, punctuation mark or symbol. We assume the sequence is both syntactically valid and semantically coherent. A *substring* of D is a contiguous sequence of tokens $\langle w_i, \ldots, w_j \rangle$ for $1 \leq i \leq j \leq |D|$.

Contradictions. Given documents D and D', a *contradiction* arises if there exist substrings $s \subseteq D$ and $s' \subseteq D'$ that refer to the same entity or relationship o, but assign mutually exclusive values to at least one attribute or property of o under the domain semantics. If $D = D'$, it is *intra-document*. If $D \neq D'$, it is *inter-document*.

Prior research in the NLP community [79, 102, 109] classifies contradictions into seven distinct types, with examples shown in Table 1: (1) *Factual*: contradictions requiring external world knowledge to verify; (2) *Content* (misinformation, emotion, opinion, semantic, and lexical types): conflicts in the attributes of entities or relationships; (3) *Causal*: contradictions where the result does not align with the cause under a clear causal relationship; (4) *Negation*: explicit negation or antonymy (*e.g.,* use of "not," "never," or antonyms); (5) *Relational*: conflicts in entity relationships, such as reversed, missing, or mismatched links; (6) *Numeric*: contradictions involving mismatched or out-of-range numbers; and (7) *Temporal*: contradictions caused by conflicting statements across different time points.

Detection. Given a document collection $\mathcal{D}$, *contradiction detection* identifies all contradictory substring pairs (s, s') where $s \in D$ and $s' \in D'$, for $D, D' \in \mathcal{D}$, and outputs their contradiction types T.

Research on contradiction detection spans social media [75, 76, 78], financial reporting [24] and biomedical literature [5, 6, 100, 102]. Early studies centered on sentence-level contradictions framed as text classification or NLI tasks [14, 20, 22, 56], Later work expanded to event-level and dialogue/QA scenarios [23, 48, 67, 93, 135]. Recent efforts push toward document-level contradiction detection, combining zero-shot inference, multi-hop reasoning and LLM-based inference [79, 90, 104, 109, 130, 131]. Benchmarking of GPT-3.5, GPT-4, PaLM-2 and LLaMA-2 shows that GPT-4 currently leads in accuracy and evidence localization [79]. Yet significant gaps remain: LLMs still face limited interpretability, frequent hallucinations, and difficulty in detecting subjective or long-range contradictions [79, 109].

Unlike prior methods, this work proposes a graph-based paradigm for textual contradiction detection: (1) entities and relationships of interest are extracted into graphs, enabling uniform detection of diverse contradictions over both vertices and edges; (2) graphs are "joined" together and enriched with domain-specific knowledge, capturing domain semantics and uniformly catching both intra- and inter-document contradictions; (3) vertices and edges are bound to substrings and coupled with NLP predicates, handling linguistic variation and ambiguity; (4) logic and ML predicates are integrated to reason about inconsistencies, account for context dependence and temporal dynamics, and provide interpretable logical explanations; (5) the process can be conducted in PTIME.

Inconsistency detection. Inconsistency detection has been extensively studied in relational [4, 9, 11, 21, 32, 33, 39, 41, 58, 88, 111, 117] and graph data [34, 36, 37, 40, 42]. Early efforts focused on consistent query answering and constraint-based repair [9, 21, 32, 33], and later extended to ML-based conflict detection and entity resolution [4, 39, 58, 88, 111]. For graphs, research explored dependencies, association rules, rule discovery and data repair [34, 37, 40, 42].

Data cleaning has also been integrated into ML training pipelines, including interactive tools [44], AutoML systems [46], provenance debugging [47, 124], differentiable preprocessing [60, 80], iterative repair [72, 73], boosting-based [71], fairness-aware [112], weak supervision [98], and self-supervised methods [85]. Recent work unifies cleaning with integration, fairness, and federated learning [54, 68, 112], and assesses its influence on ML performance [81].

DocChecker differs from prior studies as follows. (a) Unlike traditional methods for relational tables or semi-structured graphs, it ad-

Types	Sentence 1	Sentence 2	RICOs
(1) Factual	"The Nobel Prize was indeed established in 1901 to commemorate Alfred Nobel"	"Hinton received the Nobel Prize in 2024, which was established in 1901 to honor Shannon's contributions to the field of information."	Rule φ_1
(2) Content	"William was engaged while reading the paper on computer architecture"	"William found the content of the paper on computer architecture to be extremely boring"	Rule φ_2
(3) Causal	"David's video, which was published on YouTube in 2024, was permanently removed that year due to violence."	"Taylor came across this video published by David while browsing YouTube in 2025."	Rule φ_3
(4) Negation	'On April 20, 2025, the router experienced a brief routing disruption at 2:00 PM"	"The router did not report any routing disruptions throughout April 20, 2025."	Rule φ_4
(5) Relational	"The owner of the brand store, Howard, and his wife Sheila have a harmonious relationship after marriage."	"Howard and his wife Emma have a beloved little daughter named Sheila"	Rule φ_5
(6) Numeric	"Howard's brand store sold 29 pairs of women's high heels on June 15, 2025."	"Howard is concerned about the lack of any sales on June 15, 2025."	Rule φ_6
(7) Temporal	"Around 1860, Nobel and other researches believed that nitroglycerin could not treat heart disease"	"After Nobel's death, further research revealed that nitroglycerin is effective for treating heart disease"	Rule φ_7

Table 1: Examples of Contradictions and Corresponding Detection Rules

dresses textual contradictions, resolving conflicts across statements, sections and documents while handling linguistic ambiguity absent in earlier work. (b) Extending graph cleaning rules (GCRs) [31, 36], it models properties on both vertices and edges, supports edge predicates and temporal relations, and integrates NLP tools. (c) It uniformly detects intra- and inter-document contradictions by leveraging graph join and fusion to integrate multi-document evidence and trace detected conflicts back to their textual sources. (d) We also develop new rule discovery methods. Since prior rule discovery approaches are ineffective for sparse tasks like contradiction detection, we propose new generation, evaluation and validation strategies for learning rules effectively in sparse settings (Sections 1 and 6).

3 CONTRADICTION IDENTIFYING RULES

This section first extends the notion of dual patterns of [31] (Section 3.1), then defines RICO (Section 3.2), and finally settles the complexity of contradiction detection with RICOs (Section 3.3).

3.1 Dual Graph Patterns

Graphs. A *graph* is $G = (V, E, L, F_A)$, where (a) V is a finite set of vertices; (b) E is a finite set of labeled edges, with each $e = (v, l, v') \in E$ denoting an edge from v to v' labeled l; (c) L assigns a label to each vertex; and (d) F_A assigns to each vertex $v \in V$ (resp. edge $e \in E$) a tuple of attributes $(A_1 = a_1, \ldots, A_n = a_n)$, written as $v.A_i = a_i$ (resp. $e.A_i = a_i$). Unlike relational data, vertices and edges may carry heterogeneous attributes without a fixed schema.

As opposed to graphs studied in prior work [31, 34, 36, 42] where only vertices are associated with attributes, we consider graphs in which both vertices and edges carry attributes (properties).

Paths. A *path* ρ from vertex v_0 in a graph $G = (V, E, L)$ is a sequence $(v_0, l_0, v_1, \ldots, v_{n-1}, l_{n-1}, v_n)$ such that $(v_{i-1}, l_{i-1}, v_i) \in E$ for all $i \in [1, n]$. We focus on *simple* paths, where no vertex repeats. For an edge $(u, l, v) \in E$, v is a *child* of u, and u a *parent* of v.

Stars. A *star pattern* is $Q[x_0, \bar{x}, \bar{e}_x] = (V_Q, E_Q, L_Q)$, where (1) V_Q (reps. E_Q) is the set of pattern vertices (resp. edges); (2) L_Q assigns labels to vertices in V_Q; (3) $\bar{x}$ and $\bar{e}_x$ are lists of distinct vertex and edge variables, each uniquely denoting an element in V_Q and E_Q; (4) $x_0 \in \bar{x}$ is the designated *center*; and (5) Q is *star-shaped*: each vertex $x \in \bar{x}$ has a unique path from x_0 and, if $x \neq x_0$, at most one child.

Intuitively, $Q[x_0, \bar{x}]$ is a star-shaped pattern centered at entity x_0, whose leaf vertices represent characteristic features of x_0.

Dual patterns. A *dual pattern* is $\langle Q_x[x_0, \bar{x}, \bar{e}_x], Q_y[y_0, \bar{y}, \bar{e}_y]\rangle$, denoted $\boldsymbol{Q}[x_0, y_0]$, where Q_x and Q_y are disjoint star patterns. Intuitively, $\boldsymbol{Q}$ couples two entities x_0 and y_0, each with its own feature leaves, so that their properties can be compared pairwise.

We refer to a star-shaped dual star pattern simply as a *dual pattern* or a *pattern* when it is clear in the context.

Matches. A *match of a star pattern Q in a graph G* is a homomorphism h from the pattern vertices and edges in Q to G such that (a) for every $u \in V_Q$, $L_Q(u) = L(h(u))$; and (b) for every edge $(u, l, u') \in E_Q$, $(h(u), l, h(u')) \in E$ holds in G. Similarly, a *match of a dual pattern* $\boldsymbol{Q}[x_0, y_0] = \langle Q_x, Q_y\rangle$ in G is a homomorphism h from $V_{Q_x} \cup V_{Q_y}$ and $E_{Q_x} \cup E_{Q_y}$ to V and E of G, respectively.

Observe that $h(x_0)$ and $h(y_0)$ may be disconnected in G, which enables comparison of entities without direct links and with differing topological structures in a schemaless graph.

Example 1: Figure 1 depicts seven dual patterns $\boldsymbol{Q}_1$–$\boldsymbol{Q}_7$, each with two centers (x_0, y_0) and multiple paths. For example, in $\boldsymbol{Q}_1$, a path is (y_0, honor, y_1); and in $\boldsymbol{Q}_5$, a path is $(x_0, \text{owner}, x_1, \text{wife}, x_2)$. □

It is in PTIME to check whether a dual pattern $\boldsymbol{Q}$ has a match in a graph G [36]. In contrast, it is NP-complete to decide whether there exists a match of generic patterns in G (cf. [49]).

3.2 Rules for Identifying Contradictions

We next define RICOs, starting with their predicates.

Predicates. A *predicate* q of RICOs over a document collection $\mathcal{D}$ and a dual pattern $\boldsymbol{Q}[x_0, y_0] = \langle Q_x[x_0, \bar{x}, \bar{e}_x], Q_y[y_0, \bar{y}, \bar{e}_y]\rangle$ is:

$$q \quad ::= \quad S(x, s) \mid S(e_x, s) \mid \mathcal{M}(x.\bar{A}, y.\bar{B}) \mid \mathcal{M}(e_x.\bar{A}, e_y.\bar{B}) \mid x.A \oplus y.B \mid z.A \oplus c \mid e_x.A \oplus e_y.B \mid e_z.A \oplus c \mid T(s, s', \tau) \mid \tau(x.A, y.B) \mid \tau(e_x.A, e_y.B).$$

Here s is a *string variable* for a substring in a document; $x \in \bar{x}$ and $y \in \bar{y}$ (resp. $e_x \in \bar{e}_x$ and $e_y \in \bar{e}_y$) are *vertex variables* (resp. *edge variables*) in Q_x and Q_y, respectively; $z \in \bar{x} \cup \bar{y}$ (resp. $e_z \in \bar{e}_x \cup \bar{e}_y$); c is a constant in U; A and B are attributes in Υ; τ is a contradiction type such as negation or casual; and $\oplus$ is in $\{=, \neq, <, \leq, >, \geq\}$.

The predicates are classified as follows:

- Substring binding: $S(x, s)$ (resp. $S(e_x, s)$) binds a substring s to a vertex x (resp. edge e_x).
- Textual classifier: $T(s, s', \tau)$ is a NLP/NLI/LLM model that classifies whether substrings s and s' are in contradiction of type τ.
- ML predicate: $\mathcal{M}$ is a pretrained ML model returning a Boolean value (*e.g.*, prediction strength above a threshold), where $x.\bar{A}$

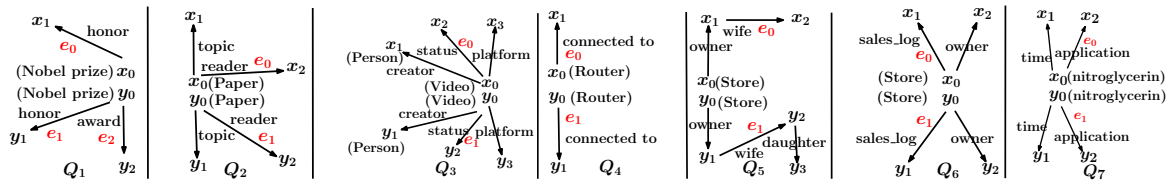


Figure 1: Example dual patterns

(resp. $e_x.\bar{A}$) denotes a list of attributes on vertex x (resp. edge e_x).

- Comparison: $x.A \oplus y.B$ and $z.A \oplus c$ (resp. $e_x.A \oplus e_y.B$, $e_z.A \oplus c$) compare values of attributes, where $\oplus \in =, \neq, <, \leq, >, \geq$.
- Type: $\tau(x.A, y.B)$ is true if and only if (iff) $x.A$ and $y.B$ are in contradiction of type τ (similarly for $\tau(e_x.A, e_y.B)$).

Textual classifiers $T(s, s', \tau)$ are pretrained NLP/NLI/LLM models, *e.g.,* (1) SBERT [99] detects attribute-level and negation/antonym contradictions; (2) NumNet [97] handles numeric reasoning; (3) knowledge_graph_maker [29] extracts relations for relational contradictions; (4) Flan-T5-base [17] captures temporal dynamics; (5) GPT-5.2 with retrieval [94] addresses factual contradictions; and (6) Prompted GPT-5.2 [94] detects causal and broad content, misinformation, emotion, opinion, semantic and lexical inconsistencies.

ML predicates $\mathcal{M}(x.\bar{A}, y.\bar{B})$ and $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$ can be implemented as pretrained graph-based models, including (1) graph-based NLI classifiers (*e.g.,* GraphSAGE [55], GAT [116]) that embed structural information to detect entailment, neutrality, and contradiction between vertices and edges, (2) graph semantic models (*e.g.,* GraphBERT [134], GCN [69]) that capture event similarity and contextual dependencies among vertices and edges, and (3) graph-based prompt-driven models *e.g.,* HGT [64]) that extend transformers to infer temporal and causal relationships in graphs.

Rules. A Rule for Identifying COntradictions (RICO) φ has the form:

$$Q[x_0, y_0](X \rightarrow q_0),$$

where $Q[x_0, y_0] = \langle Q_x[x_0, \bar{x}, \bar{e}_x], Q_y[y_0, \bar{y}, \bar{e}_y] \rangle$ is a dual pattern, X is a conjunction of predicates and q_0 is a predicate, both over $\mathcal{D}$ and Q. Moreover, (1) each binary predicate in X is defined on either two leaves x and y in Q_x and Q_y or on their incident edges e_x and e_y, and (2) each leaf and its edge carry at most one binary predicate.

We refer to Q and $X \rightarrow q_0$ as the *pattern* and *dependency* of φ, and to X and q_0 as *precondition* and *consequence* of φ, respectively.

Intuitively, restrictions (1) and (2) above balance expressivity with efficiency following [36] (see Section 3.3). We separate Q and X to (a) visualize the topological features of x_0 and y_0 via Q, and (b) accelerate evaluation by exploiting the locality of pattern matching.

Example 2: We illustrate how learned RICOs detect the seven contradiction types in Table 1, using the dual patterns in Figure 1.

(1) *Factual contradiction.* RICO φ_1: $Q_1[x_0, y_0]$ (x_0.title = Nobel Prize $\wedge$ y_0.title = Nobel Prize $\wedge$ $\mathcal{M}(x_0$.description, y_0.description) $\wedge$ y_2.name = Hinton $\wedge$ e_2.time = 2024 $\wedge$ e_2.award field = physics $\wedge$ x_1.name = Nobel $\wedge$ y_1.name = Shannon $\wedge$ $S(e_0, s) \wedge S(e_1, s') \wedge T(s, s',$ factual) $\rightarrow$ y_1.name = Nobel). This rule identifies and corrects the factual contradiction in case (1) of Table 1. The contradiction arises because the two awards have nearly identical descriptions (detected by the NLI model $\mathcal{M}(x_0$.description, y_0.description)) but list different honorees (x_1.name = Nobel vs. y_1.name = Shannon). Using external knowledge and LLM-based query results ($T(s, s',$ factual)), the rule verifies that "Nobel" is the correct honoree (y_1.name = Nobel).

(2) *Content contradiction.* RICO φ_2: $Q_2[x_0, y_0]$ ($S(e_0, s) \wedge S(e_1, s') \wedge \mathcal{M}(x_1$.field, y_1.field) $\wedge$ x_2.name = William $\wedge$ y_2.name = William $\wedge$ e_0.rating $\neq$ e_1.rating $\rightarrow$ emotion(e_0.rating, e_1.rating)). This rule finds an emotion contradiction on edge attributes (case (2) of Table 1), where the same reader (x_2.name = William $\wedge$ y_2.name = William) has conflicting comments (engaged versus boring) on the same paper (identified by $\mathcal{M}(x_1$.field, y_1.field)).

(3) *Causal contradiction.* RICO φ_3: $Q_3[x_0, y_0]$ ($S(x_2, s) \wedge S(y_2, s')$ $\wedge$ x_1.name = David $\wedge$ y_1.name = David $\wedge$ x_3.name = y_3.name $\wedge$ x_2.details = removed $\wedge$ e_0.removed_time $\leq$ e_1.viewing_time $\wedge$ y_2.details = viewable $\rightarrow$ causal(x_2.details, y_2.details)). The rule catches a causal contradiction (case (3) of Table 1), as a video cannot be accessible (y_2.details = viewable) once it has been removed (x_2.details = removed $\wedge$ e_0.removed_time $\leq$ e_1.viewing_time).

(4) *Negation contradiction.* RICO φ_4: $Q_4[x_0, y_0]$ ($S(e_0, s) \wedge S(e_1, s') \wedge$ x_1.ip = y_1.ip $\wedge$ e_0.date = e_1.date $\wedge$ e_0.interrupt = true $\wedge$ e_1.interrupt = false $\rightarrow$ negation(e_0.interrupt, e_1.interrupt)). This rule catches a negation contradiction on edge properties (case (4) of Table 1). Here Q_x identifies that the router reported an interruption (e_0.interrupt = true) at 2:00 PM on April 20, 2025, whereas Q_y indicates no interruption (e_1.interrupt = false) on the same day.

(5) *Relational contradiction.* RICO φ_5: $Q_5[x_0, y_0]$ ($S(e_0, s) \wedge S(e_1, s')$ $\wedge$ x_1.name = y_1.name $\wedge$ x_2.name $\neq$ y_2.name $\wedge$ e_0.time = e_1.time $\wedge$ $T(s, s',$ relation) $\rightarrow$ relational(e_0.spouse, e_1.spouse)). This rule catches a relational contradiction that Howard cannot be married to both Sheila and Emma at the same time (case (5) of Table 1). Note that classifier $T(s, s',$ relation) captures context dependence: the contradiction becomes clear only when the contexts of Q_x and Q_y are considered together, showing that Howard cannot have two different wives simultaneously (e_0.time = e_1.time).

(6) *Numeric contradiction.* RICO φ_6: $Q_6[x_0, y_0]$ ($S(e_0, s) \wedge S(e_1, s') \wedge$ x_2.name = y_2.name $\wedge$ e_0.sold items = women's high heels $\wedge$ e_0.sale quantity = 29 $\wedge$ e_1.sold items = women's high heels $\wedge$ e_1.sale quantity = 0 $\wedge$ e_0.date = e_1.date $\rightarrow$ numeric(e_0.sale_quantity, e_1.sale_quantity)). The rule catches a numerical contradiction on the same item that was sold by a store on the same day: sentence 1 reports 29 women's high heels were sold, whereas sentence 2 claims 0 (case (6) of Table 1).

(7) *Temporal contradiction.* RICO φ_7: $Q_7[x_0, y_0]$ ($S(x_2, s) \wedge S(y_2, s') \wedge \mathcal{M}(x_1$.time, y_1.time) $\wedge$ x_2.treatment $\neq$ y_2.treatment $\rightarrow$ temporal(x_2.treatment, y_2.treatment)). Here $\mathcal{M}$ is an ML model that determines chronological ordering, returning true if x_1.time precedes y_1.time, *e.g.,* Flan-T5-base model. In this case, x_1.time is around 1860, while y_1.time falls after Nobel's death. The rule captures a temporal contradiction: while sentence 1 states that nitroglycerin could not treat heart disease during Nobel's lifetime, sentence 2 claims it was effective after his death (case (7) of Table 1). □

Semantics. Consider RICO $\varphi = Q[x_0, y_0](X \rightarrow p_0)$. A *valuation* of φ is a match h of $Q[x_0, y_0]$ in graph G, extended with bindings of string variables to substrings in document collection $\mathcal{D}$.

We say that valuation h *satisfies* a predicate q, written $h \models q$, if (a) $q = S(x, s)$: vertex $h(x)$ is bounded to substring $h(s)$ in $\mathcal{D}$ (similarly for $q = S(e_x, s)$); (b) $q = x.A \oplus y.B$: $h(x).A \oplus h(y).B$ holds (similarly for $z.A \oplus c$, $e_x.A \oplus e_y.B$ and $e_z.A \oplus c$); (c) $q = \mathcal{M}(x.\bar{A}, y.\bar{B})$: the ML model $\mathcal{M}$ predicts true on features $h(x).\bar{A}$ and $h(y).\bar{B}$ (similarly for $q = \mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$); (d) $q = T(s, s', \tau)$: the textual classifier T assigns contradiction type τ to substrings $h(s)$ and $h(s')$; (e) $q = \tau(x.A, y.B)$: the contradiction between vertex attributes $h(x).A$ and $h(y).B$ is classified as type τ (similarly for $q = \tau(e_x.A, e_y.B)$).

For precondition X, we write $h \models X$ if valuation h satisfies *all* the predicates in X. We write $h \models X \rightarrow q_0$ if $h \models X$ implies $h \models q_0$. We say that graph G and documents $\mathcal{D}$ *satisfy* a RICO φ, written $(G, \mathcal{D}) \models \varphi$, if for all valuations h of $Q[x_0, y_0]$ over G and $\mathcal{D}$, $h \models X \rightarrow q_0$. Similarly, $(G, \mathcal{D}) \models \Sigma$ if $(G, \mathcal{D}) \models \varphi$ for every $\varphi \in \Sigma$.

Remark. Below we compare RICOs with previous graph rules.

(1) Expressivity. RICOs explicitly account for linguistic variation and ambiguity through substring bindings and textual classifiers, bridging structured graph reasoning with unstructured natural language text, which is not addressed in prior graph rule frameworks.

Unlike earlier graph rules such as GARs [34] and GCRs [31, 36], which operate on graphs where only vertices carry properties, RICOs are defined on property graphs where both vertices and edges may have attributes. They further support: (a) edge predicates ($e_x.A \oplus e_y.B$, $e_z.A \oplus c$, $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$); (b) textual classifiers $T(s, s', \tau)$; and (c) substring bindings $S(e_x, s)$. Thus, RICOs detect contradictions on both entities (vertices) and relationships (edges), whereas GARs/GCRs capture only entity-level conflicts.

(2) Pattern structure. Like GCRs, RICOs adopt extended star-shaped dual patterns, enabling efficient pairwise comparison of entity (or relationship) features. In contrast, GFDs [42] and GARs [34] are defined on generic graph patterns, which offer generality but are less tractable for valuation finding and contradiction detection.

3.3 The Complexity of Contradiction Detection

We study the *validation problem*, stated as follows.

- Input: A graph G, a document collection $\mathcal{D}$, and a set Σ of RICOs.
- Question: Does $(G, \mathcal{D}) \models \Sigma$?

This problem studies the complexity of contradiction detection.

The validation problem is coNP-hard for GARs [34] but solvable in PTIME for GCRs [31]. Despite being more expressive than both (by supporting edge predicates, substring bindings, and textual classifiers), RICOs still preserve tractability. This contrasts with general graph rules on arbitrary patterns, which are typically intractable. Thus, RICOs strike a balance between expressiveness and efficiency, enabling scalable contradiction detection.

Theorem 1: *The validation problem is in* PTIME *for* RICOs. □

Proof sketch: An algorithm for RICO validation proceeds as follows: for each φ, it checks whether there exists a valuation h of Q over $(G, \mathcal{D})$ such that $h \models X$ but $h \not\models q_0$. (1) It first locates matches of leaves satisfying X but violating q_0. (2) It then deduces matches of other vertices in a bottom-up manner. The procedure runs in PTIME, since evaluating predicates such as $S(x, s)$, $T(s, s', \tau)$, $\mathcal{M}(x.\bar{A}, y.\bar{B})$, $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$, $\tau(x.A, y.B)$ and $\tau(e_x.A, e_y.B)$ is polynomial, and star-pattern matching is also in PTIME [31]. □

4 AN OVERVIEW OF THE SYSTEM

This section outlines DocChecker, our system for detecting contradictions. Given a document collection $\mathcal{D}$, DocChecker identifies all contradictions either within a single document or across documents. Users may optionally provide a set of target entities $\mathcal{E}$ and a domain knowledge graph KG to enhance semantic grounding. As shown in Figure 2, DocChecker operates through five sequential stages.

(1) Graph conversion. For each document D in $\mathcal{D}$, DocChecker converts D into a graph G_D using pretrained ML models and LLM, extracting only vertices and edges for entities in $\mathcal{E}$ to exclude irrelevant objects. It records each entity's position in D so that contradictions in G_D can be mapped back to the text. Logically, it defines a relation $S(x, s)$ linking vertices to substrings; physically, this yields a mapping f_{index} from G_D's vertices to entity positions in D.

In order to incorporate specialized knowledge in the domain and enrich the converted graph, DocChecker joins each G_D and knowledge graph KG when available. The join is conducted only on entities in $\mathcal{E}$, *i.e.,* it extracts from KG only relevant knowledge of users' interest. This yields an enriched graph, also denoted as G_D.

(2) Graph fusion. To synthesize the interplay of multiple statements across sentences, sections and documents, DocChecker fuses all graphs G_D into a single graph G. It is carried out by computing the "join" of G_D's, such that vertices across different graphs are "connected" in G if they refer to the same real-world entity.

(3) Rule discovery. Taking a sample of the fused and enriched graph G, DocChecker learns a set Σ of high-quality RICOs with document-specific patterns and domain-specific context, in parallel.

(4) Contradiction detection. DocChecker then applies the set Σ of RICOs to G in parallel, to detect contradictions of vertices and edges in G, intra-document and inter-document in a unified process. It guarantees the parallel scalability [74] to scale with large $\mathcal{D}$.

(5) Document marking. Using the mapping f_{index} from graphs to documents, DocChecker maps the detected contradictions back to the documents of $\mathcal{D}$, by marking the corresponding text in position.

Below we outline the process of steps 4 and 5 (see details in [1]). We will present steps 1-2 and 3 in Sections 5 and 6, respectively.

Contradiction detector. Step 4 of DocChecker detects conflicts by chasing graph G with RICOs Σ, called Detector (see [1] for details).

Facts. For each attribute $x.A$ (resp. $e_x.A$), the system maintains relations $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$), where $\otimes \in \{=, \neq, <, \leq, >, \geq\}$. Each $[x.A]_\otimes$ records the attributes and constants that satisfy the $\otimes$ relation with $x.A$, as well as contradiction-type labels; these are derived from a set Γ containing both the initial ground truth and facts generated during the chase. All $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) are valid, *e.g.,* no conflicting constants $c_1 \neq c_2$ may coexist in $[x.A]_=$.

Ground truth. Initial facts in Γ are validated by users or experts.

The chase. For a RICO $\varphi : Q[x_0, y_0](X \rightarrow q_0)$ and valuation h satisfying premise X, if current facts $\mathcal{F}$ validate X, consequence q_0 is added to $[x.A]_\otimes$, yielding extended set $\mathcal{F}'$. Formally, a *chase step* of graph G with Σ at a set $\mathcal{F}$ of facts is $\mathcal{F} \Rightarrow_{(\varphi,h)} \mathcal{F}'$, which applies φ under the valuation h to produce the updated fact set $\mathcal{F}'$. That is,

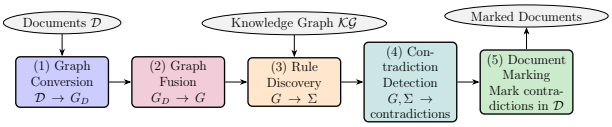


Figure 2: The workflow of DocChecker

the chase expands ground truth with validated facts in the process. A *chasing sequence* ξ is a sequence $\mathcal{F}_0, \ldots, \mathcal{F}_n$, where $\mathcal{F}_0 = \Gamma$ and each $\mathcal{F}_{i-1} \Rightarrow_{(\varphi,h)} \mathcal{F}_i$ is valid, *i.e.,* it introduces no conflicts.

Termination. Chase ξ terminates when no rule extends $\mathcal{F}_n$. When ξ terminates, we refer to the set $\mathcal{F}_n$ of facts as the result of ξ.

Church-Rosser property. The chase ensures deep contradiction detection. Under correct rules and accurate predicates, only correct facts are deduced. Further, the chase is Church-Rosser, *i.e.,* the fixpoint is unique regardless of the order in which rules are applied [2].

Proposition 2: *The chase over G with Σ is Church-Rosser.* □

Proof sketch: The proof has two steps. (1) *Finiteness*: Chase steps add facts to a finite number of attributes, bounded by $O(|\Gamma||G| + |G|^2)$. (2) *Convergence*: If two terminal results differ, a missing fix would apply to both, contradicting terminality (see [1]). □

Parallel chasing. Detector implements the chase using the Bulk Synchronous Parallel (BSP) model [115], using one master and K workers. The graph G and document collection $\mathcal{D}$ are partitioned across K workers: (1) each document graph G_D and its document D are colocated on the same worker; (2) all dual-star patterns $Q[x_0, y_0]$ sharing the same x_0 (resp. y_0) and their contexts are placed in the same partition to ensure locality; (3) when G_D or D is large, the associated $Q[x_0, y_0]$ and contexts are further partitioned for load balancing; and (4) each rule φ is dispatched only to the workers containing its corresponding $Q[x_0, y_0]$. Workers execute in parallel, and the master aggregates all newly derived $[x.A]_=$ (resp. $[e_x.A]_=$) into Γ.

We show that the algorithm is *parallelly scalable* [74]: its runtime is $t(\mathcal{D}, G, \Sigma)/K$, where $t(\mathcal{D}, G, \Sigma)$ is the cost of a yardstick sequential algorithm for contradiction detection. Thus its parallel runtime scales approximately inversely with K: as the number of processors increases, the cost decreases nearly linearly (see [1] for proof).

Document marking. To map graph-level contradictions in G back to their textual origins in $\mathcal{D}$, step 5 of DocChecker maintains, for each triple (h, r, t) in every dual star $Q[x_0, y_0]$, a mapping $f_{\text{index}}(u) : u \mapsto [s_u, e_u]$, where $u \in \{h, t\}$ and $[s_u, e_u]$ is the sentence span in $\mathcal{D}$ containing the entity mention. These spans provide the exact text context for annotating contradictions. If a statement conflicts with multiple others, all involved spans are marked via $f_{\text{index}}(u)$, enabling fine-grained multi-point conflict tracking.

5 GRAPH CONVERSION AND FUSION

This section presents graph conversion (Section 5.1) and graph fusion (Section 5.2), *i.e.,* steps 1-2 of DocChecker, respectively.

5.1 Graph Conversion

Graph conversion transforms each document $D \in \mathcal{D}$ into a graph G_D via LLM-based generation, and enriches the resulting vertex and edge attributes using an external knowledge graph (KG).

LLM-based graph conversion. Given a document $D \in \mathcal{D}$ and a user-specified entity set $\mathcal{E}$, DocChecker converts parts of D that involve $\mathcal{E}$ into a property graph G_D, whose edges capture relationships among entities in $\mathcal{E}$, in two steps. First, a pretrained NER model [133] identifies all mentions of $\mathcal{E}$'s entities in D. Second, these mentions and their contexts in D are formatted as prompts and used to instruct an LLM (GPT-5.2 [94]), which generates a property graph with appropriate relationships, labels and attributes.

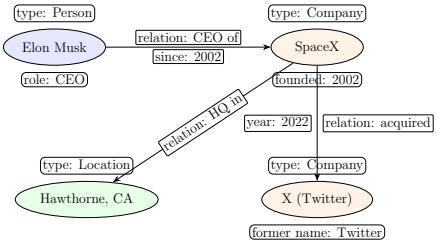


Figure 3: Example graph G_D

Example 3: Given document D *"Elon Musk, the CEO of SpaceX (founded in 2002), acquired X (formerly Twitter) in 2022. SpaceX, a private company, has its headquarters in Hawthorne, CA"* and the entity set $\mathcal{E}$ = {Elon Musk, SpaceX, X (Twitter), Hawthorne, CA}, the NER model extracts the relevant contexts: (1) Elon Musk, the CEO of SpaceX, (2) SpaceX (founded in 2002), acquired X (formerly Twitter) in 2022, and (3) SpaceX has its headquarters in Hawthorne, CA. These contexts are formatted as a prompt (given in [1]) and used to instruct the LLM, which generates the property graph G_D in Figure 3, capturing the entities and their relationships. □

Graph enrichment. To recover information omitted by LLMs during graph generation [65, 95, 120] and to supplement documents D that are not self-contained in specialized domains, we enrich each generated graph G_D with knowledge from a curated KG (*e.g.,* Wikidata). Given G_D, the KG, and entity set $\mathcal{E}$, we augment G_D with domain-relevant facts about entities in $\mathcal{E}$. Enrichment consists of two steps: (1) entity linking and disambiguation, and (2) selective extraction of KG attributes and relations, combining efficient retrieval with context-based disambiguation and no manual effort.

Entity Linking. We adopt an automatic two-stage strategy to map each entity node $e_i \in \mathcal{E}$ in G_D to its unique identifier q_i in the KG.

(1) Candidate retrieval. For each entity e_i, the Wikidata API (wbsearchentities) is queried automatically via standardized templates [118], which supports high throughput [89]. This returns a candidate set $Q_i = \{q_1, \ldots, q_k\}$ of the top-k most relevant identifiers ($k = 10$ yields recall of over 90% [28, 61]). Each candidate is attached with its description and aliases from the KG. This step restricts the search space to a compact and accurate set, which provides high scalability to thousands of entities per document [106].

(2) Contextual disambiguation. Then, the best match of e_i from Q_i is selected as follows. Taking $C(e_i)$ and $\text{Desc}(q_j)$ as inputs, a SBERT model [99] embeds a context vector $\mathbf{v}_{C(e_i)}$ for e_i and a description vector $\mathbf{v}_{\text{Desc}(q_j)}$ for each candidate q_j in Q_i, where $C(e_i)$ is the KG descriptions for the two-hop neighboring entities of e_i, and $\text{Desc}(q_j)$ is the description of q_j. Then the best matching identifier q^* for e_i is selected with the highest cosine similarity of vectors: $q^* = \arg\max_{q_j \in Q_i} \frac{\mathbf{v}_{C(e_i)} \cdot \mathbf{v}_{\text{Desc}(q_j)}}{\|\mathbf{v}_{C(e_i)}\| \, \|\mathbf{v}_{\text{Desc}(q_j)}\|}$. Thus, the system resolves ambiguities without manual review. Note that only neighbors within two-hops are considered since they provide enough information for disambiguation with limited cost [28, 52].

Selective extraction. Given G_D with linked KG identifiers, attributes

and relations are retrieved via SPARQL to enrich the graph. To ensure relevance, a domain label (*e.g.,* finance, biomedicine) is inferred for each entity using GPT-5.2 [94]. Retrieval is also restricted to a two-hop neighborhood to be efficient and context-aware, and only domain-specific predicates are retained. Selective extraction preserves essential evidence while reducing noise.

After these two stages, we obtain the enriched graph (also denoted by G_D), which encodes D with augmented KG knowledge.

Example 4: Consider the node John_Smith labeled Person in G_D. The enrichment first links the node to Wikidata entity Q123456 and retrieves attributes occupation : Software_Engineer, citizenship : United_States and educated_at : Stanford_Univ. These attributes are attached to the John_Smith, enabling richer semantic reasoning (*e.g.,* matching by education or nationality). □

Complexity. Graph conversion comprises three steps: (1) NER-based context extraction, (2) LLM-based graph generation, and (3) KG enrichment. Let N be the number of documents; L_D the average document length; L_P and L_G the prompt and generated-graph lengths; and $|\mathcal{E}|$ the number of entities to enrich. The total cost is $O(NL)$, where $L = L_D + L_P + L_G + |\mathcal{E}|$. Specifically, NER costs $O(NL_D)$, graph generation costs $O\big(N(L_P + L_G)\big)$, and enrichment costs $O(N|\mathcal{E}|)$. The documents are independent and thus are processed in parallel, running in $O(NL/K)$ on K processors under balanced load.

5.2 Graph Fusion

Graph fusion identifies nodes in different G_D's that refer to the same real-world entity, links them with a special ϵ-edge, and merges all G_D's into a unified graph G. Below we first revise the notion of graph join [35] and then extend it to introduce the graph fusion.

Graph join. Graph join identifies entities referring to the same real-world object [35]. Prior methods merge such entities into a single node by unifying their attributes and relations [26], which obscures the provenance of attributes and relations and makes it difficult to trace contradictory facts back to their source documents. Hence we develop a provenance-preserving hybrid alignment method.

Local graph embedding. We use an HGNN-based model (*e.g.,* R-GCN [103]) to learn embeddings $\mathbf{h}_u$ and $\mathbf{h}_v$ for nodes $u \in G_{D_1}$ and $v \in G_{D_2}$, capturing both attributes and local topology. First, SBERT encodes the raw attributes A_u into an initial vector $\mathbf{a}_u$. Then, the HGNN performs structure-aware aggregation: $\mathbf{h}_u = \sigma\Big(\sum_{r\in\mathcal{R}} \sum_{v'\in\mathcal{N}_r(u)} \frac{1}{|\mathcal{N}_r(u)|} \mathbf{W}_r \mathbf{h}_{v'} + \mathbf{W}_0 \mathbf{a}_u\Big)$, where $\mathcal{R}$ is the set of relation types and $\mathbf{W}_r$ is the weight matrices.

Matching and ϵ-edges. Given the fused embeddings $\mathbf{h}_u$ and $\mathbf{h}_v$, we compute the similarity between nodes u and v via cosine similarity: $S(u, v) = \cos(\mathbf{h}_u, \mathbf{h}_v)$. If $S(u, v)$ exceeds a threshold τ, u and v are connected by an ϵ-edge rather than be merged. This preserves the distinct structures and attributes of each entity node in their respective graphs G_D, which enables fine-grained analysis of their interplay during contradiction detection. The hybrid alignment offers three benefits. (1) SBERT provides semantic similarity even when attribute texts differ literally. (2) HGNN aggregation incorporates 2-hop structural context [28], allowing alignment beyond sparse or inconsistent attributes. (3) The combination is robust to missing or heterogeneous information, linking entities that share limited context or weak structural cues.

Example 5: Consider nodes u and v from documents D_u and D_v that both refer to Elon Musk: (1) u: `{name: "Elon Musk", attribute: {"Role": "CEO"}}`, (2) v: `{name: "Elon Musk", attribute: {"Role": "CTO"}}`. Previous graph join [26] merges them into a single node:

```
w: {name: "Elon Musk", attribute: {"Role": "CEO, CTO"}}}
```

This direct join yields conflicting attributes (*e.g.,* `Role: "CEO, CTO"`). In contrast, we retain both nodes and connect them with an ϵ-edge:

```
u: {name: "Elon Musk", attribute: {"Role": "CEO"}}
v: {name: "Elon Musk", attribute: {"Role": "CTO"}}
(u, v): {label: "epsilon"}
```

Our approach preserves both nodes and links them with an ϵ-edge, such that (1) `CEO` vs. `CTO` conflict remains explicit for contradiction detection; and (2) nodes u and v stay associated with their documents D_u and D_v, which is essential for document resolution. □

Graph fusion. We merge all G_D's into a single unified graph G. This aligns entity references across documents, providing shared context for entity matching and conflict detection. Without fusion, pairwise detection on isolated G_D's may miss cross-document inconsistencies, degrading inter-document detection accuracy.

Given the collection of document-level graphs $\{G_{D_i}\}_{i=1}^{N}$, the fusion process uses a low-complexity composition strategy that links coreferent entities with ϵ-edges while preserving the structure of each G_{D_i}. First, for every pair $\langle G_m, G_n\rangle$, we identify coreferences and insert the corresponding ϵ-edges using our revised graph join. Second, we construct the global graph G by adding all nodes and edges from $\{G_{D_i}\}_{i=1}^{N}$ together with the identified ϵ-edges:

$$G \leftarrow \Big(\bigcup_{i=1}^{N} G_{D_i}\Big) \cup \{\text{all } \epsilon\text{-edges}\}.$$

Complexity. Let $n = |V_{\text{total}}|$ and $\mathcal{E}_{\text{total}}$ be the node and edge sets of the final graph G. The combined *graph join* (entity alignment) and *graph fusion* (aggregation and enrichment) steps run in $O(n \log n + n + |\mathcal{E}_{\text{total}}| + q)$ time, where $q \le n$ is the number of KG queries. Blocking (*e.g.,* hash or LSH) reduces alignment from $O(n^2)$ to $O(n \log n)$, and fusion performs a linear scan of all nodes and edges plus q constant-time lookups. Since $q = O(n)$, the total complexity becomes $O(n \log n + |\mathcal{E}_{\text{total}}|)$, excluding system-level overhead such as network latency.

6 DISCOVERING CLEANING RULES

This section presents Learner, the algorithm for learning RICOs, *i.e.,* step 3 of DocChecker. We state the discovery problem (Section 6.1), develop Learner (Section 6.2) and parallelize it (Section 6.3).

6.1 The RICO Discovery Problem

Challenge. RICO discovery poses challenge beyond traditional rule mining [30] where quality metrics, *e.g.,* support, become invalid. As each RICO targets a specific contradiction type and typically involves only a small set of entities, most useful RICOs have low support and would be pruned by conventional high support thresholds.

Evaluation metrics. We replace support with two metrics.

Contradiction coverage: $f_c(C, \Sigma) = \frac{|\{u \in C \mid \text{detected}(u, \Sigma)\}|}{|C|}$ measures the fraction of contradictions correctly detected by rule set Σ. If $\mathcal{D}$ provides fine-grained annotations (location and type), C contains

all contradiction instances $\mathcal{I}_C$, and a unit u is detected only if both its location and type are correctly identified. If $\mathcal{D}$ offers only binary labels, C consists of all contradictory segments $\mathcal{S}_C$, and u is detected if Σ correctly marks the segment as contradictory.

Accuracy improvement: $f_a(\mathcal{D}_{\text{filter}}, \mathcal{D}) = \text{Acc}(\mathcal{D}_{\text{filter}}) - \text{Acc}(\mathcal{D})$ measures the gain in binary classification accuracy after removing contradictions, where $\mathcal{D}_{\text{filter}} = \mathcal{D} \setminus \{s \in \mathcal{D} \mid \Sigma \text{ detects } s \text{ contradictory}\}$. Binary accuracy is $\text{Acc}(\mathcal{D}) = \frac{1}{|\mathcal{D}|}\sum_{s\in\mathcal{D}} \mathbb{I}(\hat{y}_s = y_s)$, where $\hat{y}_s$ and y_s are predicted and ground-truth labels, respectively, and $\mathbb{I}(\cdot)$ is the indicator function. This evaluates whether contradiction filtering improves distinguishing normal from contradictory texts.

Problem statement. Given a document set $\mathcal{D}$ and its converted, enriched, and fused graph G, the RICO discovery problem is to find a rule set Σ from G such that Σ achieves contradiction coverage at least α and accuracy improvement at least β for bounds α and β.

6.2 Discovering RICOs by Learning

Our RICO discovery algorithm has two properties. (1) It formulates RICO discovery as a learning-based optimization problem, targeting accuracy improvement while avoiding exponential search. (2) The proposed evaluation metrics effectively select useful RICOs for contradiction detection, even when such rules have low support.

Our algorithm, Learner (Algorithm 1) is learning-based. It leverages LLMs to automatically generate RICO rules satisfying the metrics in Section 6.1. It consists of three modules: (1) dual-star pattern extraction and pruning, (2) predicate generation and gold-standard construction, and (3) three-stage iterative rule discovery. Learner operates through template-based prompt automation [122], enabling a fully autonomous and scalable pipeline.

Dual-star pattern extraction and pruning. This module extracts common dual-star patterns from the document graph G and uses an LLM to prune unpromising candidates. It first extracts generalized dual-star patterns $\mathbb{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ from G (line 1) via BFS starting from entity nodes connected by ϵ-edges (labeled `same_entity`) up to depth l, *e.g.,* 5 [126]. Each pattern stores only node and edge labels and structure, without attributes such as entity names or values, providing sufficient structural context for rule matching [63] while enabling generalization across entities.

Learner refines $\mathbb{Q}$ using a two-stage pruning pipeline [27] (line 2): (1) an LLM evaluates each $Q_i \in \mathbb{Q}$ against its document context $\mathcal{D}_i$ across seven contradiction types (see [1]). (2) Low-confidence cases (< 0.5) are then routed to a Reflector LLM [107] (lines 3–5) for chain-of-thought cross-validation. Only verified contradiction patterns are retained in the final set $\mathbb{Q}_s = \{Q'_1, Q'_2, \ldots, Q'_s\}$.

Predicate generation and gold standard construction. This module generalizes attributes into typed predicates and builds the gold standard for evaluation. Given $\mathbb{Q}_s$, Learner uses an LLM to generate a predicate set $\mathcal{P} = \{P_1, \ldots, P_s\}$ (line 6), formalizing attributes and relations into typed predicates (*e.g.,* comparison predicates $z.A \oplus c$ and textual classifiers $T(s_1, s_2, \tau)$) that compose rule preconditions and consequences. For evaluation, Learner derives a gold standard set C from ground-truth annotations (line 7), including binary labels, contradiction types, and evidence spans [79]. Intuitively, C is a set of constraints specifying which entity–attribute (or relation) assertions cannot co-exist, serving as the objective reference for validating rule accuracy and coverage (see [1] for details).

Algorithm 1: Algorithm Learner

Input: Graph G, corpus $\mathcal{D}$, threshold α, objectives $f_c(\cdot)$ and $f_a(\cdot)$.
Output: Rule set Σ

1 Extract generalized dual-star patterns $\mathbb{Q} = \{Q_1, \ldots, Q_n\}$;
2 Prune $\mathbb{Q}$ via LLM with context $\mathcal{D}_i$; obtain $\mathbb{Q}_s = \{Q'_1, \ldots, Q'_s\}$;
3 **for** $Q'_i \in \mathbb{Q}_s$ **do**
4 Annotate Q'_i with confidence score;
5 **if** *score* < 0.5 **then** Reflector LLM decides whether to keep Q'_i;
6 Generate predicate set $\mathcal{P} = \{P_1, \ldots, P_s\}$ for $\mathbb{Q}_s$;
7 Extract gold standard C from $\mathcal{D}$;
8 $\Sigma \leftarrow \emptyset$; $\Sigma' \leftarrow \emptyset$; iter $\leftarrow 0$; FailCount $\leftarrow 0$;
9 **while** $f_c(\Sigma, C) \le \alpha$ ***and*** FailCount < 3 **do**
10 $\Sigma_{\text{new}} \leftarrow \emptyset$;
11 **for** $Q'_i \in \mathbb{Q}_s$ **do**
12 Generate rules Σ_i via LLM from $\mathcal{P}$;
13 Retain $\Sigma'_i \subseteq \Sigma_i$ hitting gold standard C;
14 $\Sigma_{\text{new}} \leftarrow \Sigma_{\text{new}} \cup \Sigma'_i$;
15 $\Sigma' \leftarrow \Sigma' \cup \Sigma_{\text{new}}$;
16 Generalize predicates in Σ' (e.g., Smith.age $\rightarrow$ Person.age);
17 Use generalized Σ' as few-shot prompt to guide next iteration;
18 **if** (iter + 1) mod 3 = 0 **then**
19 Apply Σ' to $\mathcal{D}$; obtain $\mathcal{D}_{\text{clean}}$;
20 **if** $f_a(\mathcal{D}_{\text{clean}}) > f_a(\mathcal{D})$ **then**
21 $\Sigma \leftarrow \Sigma \cup \Sigma'$; FailCount $\leftarrow 0$;
22 **else**
23 FailCount $\leftarrow$ FailCount + 1;
24 $\Sigma' \leftarrow \emptyset$;
25 iter $\leftarrow$ iter + 1;
26 **return** Σ;

Three-stage discovery. Given the predicate set $\mathcal{P}$ and gold standard C, Learner initializes two empty sets, Σ' (candidate rules) and Σ (final rules) (line 8), and then iteratively performs three steps.

Stage 1: LLM-based rule generation (lines 11–15). For each candidate pattern $Q'_i \in \mathbb{Q}_s$, the LLM generates an initial rule set Σ_i, using predicates in $\mathcal{P}$ to form preconditions and consequences. Rules contain at most k precondition predicates for a bound k and a single consequence [91, 95], balancing expressiveness and efficiency. Rules are applied to Q'_i, and only those that correctly match the gold standard C are retained as Σ'_i, which are accumulated into Σ'.

Stage 2: Meta-learning feedback (lines 16–17). To enhance generalization, Σ' is used as few-shot guidance. Before prompting, entity- and attribute-specific names are replaced with typed labels (*e.g.,* `Smith.age` $\mapsto$ `Person.age`). The LLM then derives generalized rule patterns, improving subsequent generations.

Stage 3: Verification (lines 18–25). Every three iterations, rules in Σ' are assessed. Detected contradictions are mapped back to get $\mathcal{D}_{\text{filter}}$. If $f_a(\mathcal{D}_{\text{filter}}) > f_a(\mathcal{D})$, rules are added to Σ and the failure counter resets; otherwise, they are discarded and the counter increments.

The process terminates when (a) Σ achieves contradiction coverage $f_c(\Sigma, C) \ge \alpha$, and (b) accuracy improvement $f_a(\mathcal{D}_{\text{filter}}, \mathcal{D}) \ge \beta$.

Example 6: To illustrate Learner, consider a medical document $\mathcal{D}$ describing patient **John Doe**, where the admission record M_1

reports age 45 and the discharge summary M_2 reports age 65.

Phase 1. Learner identifies a dual-star centered on M_1 and M_2, linked via ϵ-edges to *John_Doe*: $Record(M_1) \xrightarrow{\epsilon} Person(P) \xleftarrow{\epsilon} Record(M_2)$. From the associated text, the LLM detects a *Numerical Contradiction* and adds the pattern to $\mathbb{Q}_s$ with confidence 0.95.

Phase 2. From this contradiction, Learner generates typed predicates (P_{age}, P_{status}, and comparison $z.A \oplus c$) and instantiates textual classifiers for "45-year-old" vs. "age is listed as 65". These are aligned with the corpus annotations to form the gold standard C.

Phase 3. (1) *Generation:* The LLM constructs a rule such as $M_1.\text{age} \neq M_2.\text{age} \rightarrow \tau_{\text{num}}$, which is added to Σ' after matching C. (2) *Meta-learning:* Specific values are generalized (*e.g.*, $c_1 \neq c_2$ to $x.A \neq y.B$, for unequal attributes across records of the same entity). (3) *Verification:* After periodic evaluation, filtering contradictions improves accuracy on $\mathcal{D}_{\text{filter}}$, and the rule is accepted into Σ. □

6.3 Learning RICOs in Parallel

The parallel version PLearner of Learner accelerates the computationally intensive components of the discovery loop, while preserving sequentially dependent stages such as meta-learning feedback. As detailed in [1], PLearner introduces three parallelized modules: parallel rule generation and filtering, parallel predicate generalization, and parallel rule application and document cleaning.

Parallel rule generation and filtering. This is the primary source of speedup. PLearner distributes the candidate dual-star patterns $\mathbb{Q}_s$ across K workers (lines 5–8). Each worker processes its assigned Q'_i independently, generating rules via the LLM and filtering them using the gold standard. The retained rules Σ'_i are aggregated atomically into Σ_{new}, eliminating the latency of sequential LLM calls.

Parallel predicate generalization. This step (*e.g.*, replacing entity names with type labels) is done independently for each rule in Σ', needs no cross-worker coordination, and is parallelized (line 10).

Proposition 3: PLearner *scales parallelly relative to* Learner. □

Proof sketch: Let T_{seq} and T_{par} denote the runtimes of Learner and PLearner (with K workers). The dominant cost of Learner is rule generation: $T_{\text{seq}} \approx \sum_{i=1}^{|\mathbb{Q}_s|} t_{\text{llm}}(i)$, where $t_{\text{llm}}(i)$ is the LLM latency for pattern Q'_i. In PLearner, patterns are evenly distributed across K workers, so $T_{\text{par}} \approx \frac{1}{K} \sum_{i=1}^{|\mathbb{Q}_s|} t_{\text{llm}}(i)$, with negligible extra overhead. Thus the speedup $S = T_{\text{seq}}/T_{\text{par}} \approx K$ for large $|\mathbb{Q}_s|$ (see [1]). □

7 EXPERIMENTAL STUDY

Using real-life documents, we empirically verify DocChecker's accuracy, scalability and its performance gains on LLM tasks.

Experimental setting. We start with the setting.

Datasets. We evaluate DocChecker on 15 publicly available datasets across five tasks: (1) *Coverage* for detecting all seven contradiction types across *all* datasets. Among these datasets, ContraDoc [79], ECON [66], Knowledge Conflict [121], StorySumm [110], DOC-NLI [130], ContractNLI [70], ConTRoL [83] and CaseHOLD [136] support inter-document tasks, whereas binary-FEVER-NLI [130], MCTest-NLI [130], SNLI [14], MNLI [123], RTE-3 [51], Negation [23] and the Real-life Contradiction Corpus [23] support intra-document tasks (see [1] for details). (2) *Sensitivity analysis* using ContraDoc. (3) *Document-level self-contradiction detection (DSCD)* for binary decision, top-k ranking and evidence retrieval, using ContraDoc and ECON. (4) *Natural language inference (NLI)* for three-way classification (Entailment/Neutral/Contradiction), at both sentence-level (SNLI, MNLI) and document-level (CaseHOLD, ContractNLI). (5) *Domain-specific contradiction analysis* for complex legal documents and domain conflicts, using ECON and CaseHOLD. (6) *Scalability analysis* of Learner, Detector, and DocChecker across six datasets: CaseHOLD, ConTRoL, ContraDoc, MNLI, DocNLI, and RTE. And (7) *An ablation study* of DocChecker on StorySumm.

Baselines. We compared DocChecker with 15 recent baselines. (1) *Document-level contradiction tasks.* (a) Sensitivity analysis tasks: GPT-4o [3]. (b) DSCD tasks: GPT-5.2-chat-latest [94], Claude-3-7-sonnet-latest [8], Gemini-3-pro-preview-thinking [50], Llama-3-70B [114], Qwen-Max-Latest [10], DeepSeek-V3 [82], Longformer-base-4096 [12] and RoBERTa-large-MNLI [84]. (2) *NLI tasks.* (a) Document-level: Longformer-base-4096, BART-base [77], DeBERTa-v3-base [57], DistilBERT-base [101] and RoBERTa-large-MNLI. (b) Sentence-level: BERT-base-uncased [25], RoBERTa-base [84], XLNet-base-cased [129] and RoBERTa-large-MNLI.

Metrics. We evaluate contradiction detection, classification and evidence retrieval using: (1) *Contradiction metrics* [13, 14]: (a) Accuracy ($\text{Acc} = \frac{\text{TP+TN}}{\text{TP+TN+FP+FN}}$) to quantify the proportion of correct predictions, as a standardized metric for benchmarking our approach against NLP/ML baselines [12, 45, 57, 77, 84], where TP, TN, FP and FN denote true positives, true negatives, false positives, and false negatives, respectively; (b) Precision ($P = \frac{\text{TP}}{\text{TP+FP}}$), the ratio of true positives to all predicted positives; Recall ($R = \frac{\text{TP}}{\text{TP+FN}}$), the ratio of true positives to all actual positives; and F_1 ($= \frac{2PR}{P+R}$), the harmonic mean of precision and recall. These provide a standard and comparable basis for benchmarking against NLP/ML baselines. (2) *Evidence Hit Rate (EHR)* [113]: the probability $P(V(E) = 1 \mid J(D) = 1)$, where $J(D)$ indicates correct contradiction prediction and $V(E)$ correct evidence localization, measuring whether the model can localize contradictions once detected. (3) *Match Rate (MR)* [97]: the fraction of gold evidence tokens covered by retrieved snippets, quantifying semantic/token-level overlap with ground truth.

Data partitioning and model training. For pre-partitioned datasets, we follow the original splits. For others, we randomly divide data into training/validation/test sets with a 7:1:2 ratio. Baselines are trained, rule patterns learned, and LLM few-shot prompting performed on the training set. Hyperparameters are tuned on the validation set, and final results are reported on the test set.

Environment. DocChecker is implemented in Python. All experiments are conducted in PyTorch on a single server with four NVIDIA GPUs (32GB each) and an AMD EPYC 9754 128-core CPU. Each GPU node is allocated 24 vCPUs and 72 GB RAM, totaling 96 vCPUs and 288 GB memory. We adopt data-parallel training across all GPUs and report the average of three independent runs.

Experimental findings. We next report our findings.

Exp-1: Accuracy. We evaluate DocChecker's accuracy and robustness against SOTA LLMs and specialized models across standard benchmarks: (1) contradiction detection with location, type and count annotations (ContraDoc [79]); (2) sensitivity to contradiction position, type and domain (ContraDoc [79]); (3) DSCD [79, 109];

Dataset	Total	Factual	Content	Causal	Negation	Relational	Numeric	Temporal	Vertex	Edge	Acc	P	R	F_1	EHR
binary-FEVER-NLI	105	58	3	1	0	30	4	9	65	40	0.789	0.895	0.869	0.882	0.853
CaseHOLD	286	78	60	135	1	1	4	7	142	144	0.868	0.879	0.986	0.929	0.864
ConTRoL	229	57	19	119	1	3	18	12	94	135	0.793	0.895	0.875	0.885	0.854
ContraDOC	517	146	41	218	5	1	53	53	240	277	0.841	0.885	0.944	0.914	0.860
ContractNLI	65	3	38	21	1	1	0	1	41	24	0.878	0.878	1.000	0.935	0.865
DOCNLI	223	38	28	111	0	2	28	16	94	129	0.850	0.883	0.958	0.919	0.861
ECON	363	239	13	17	9	0	71	14	323	40	0.801	0.893	0.886	0.890	0.855
Knowledge Conflict	90	63	13	1	0	0	8	5	84	6	0.670	0.918	0.713	0.803	0.838
binary-MCTest-NLI	217	48	36	114	3	1	5	10	89	128	0.872	0.879	0.991	0.932	0.864
MNLI	1309	456	168	419	101	7	85	73	709	600	0.770	0.899	0.844	0.870	-
Negation	397	316	0	15	31	2	22	11	338	59	0.790	0.895	0.870	0.883	0.853
Real-life	131	68	4	20	2	0	22	15	94	37	0.804	0.892	0.891	0.892	0.855
RTE-3	549	245	73	120	1	9	69	32	387	162	0.836	0.886	0.937	0.911	-
SNLI	913	391	269	90	12	64	29	58	689	224	0.870	0.879	0.989	0.931	-
StorySumm	81	42	7	25	4	0	2	5	47	34	0.852	0.883	0.961	0.920	0.862

Table 2: Accurcy: Contradiction detection and evidence retrieval across all datasets

(4) document-level [66, 70, 130] and sentence-level NLI [14, 123].

(1) Contradiction detection. As shown in Table 2, we evaluate DocChecker on 15 datasets. To analyze structural contradiction patterns, we map the seven contradiction types onto our graph representation as: *Vertex-level (entity) contradictions*: conflicts within a single node, where attributes or facts about the same entity disagree. *Edge-level (relation) contradictions*: conflicts expressed along an edge, where the logical, causal, or directional relation between two entities is inconsistent. Mapping all contradiction types onto vertices or edges enables quantifying their structural distribution. Note that EHR is omitted (marked as "-") in Table 2 for datasets lacking explicit ground-truth evidence annotations (*e.g.,* MNLI, SNLI).

(a) Contradiction type distribution. Table 2 shows substantial variation in the distribution of the seven contradiction types across datasets. (i) Factual and Causal inconsistencies dominate many datasets (*e.g.,* MNLI: 456 factual, 419 causal; ContraDoc: 218 causal), and DocChecker remains stable on these categories, achieving high F_1 on SNLI (93.1%) and ContraDoc (91.4%). (ii) Relational, Numeric, and Temporal contradictions are sparser but more challenging. For instance, Relational cases appear frequently in SNLI (64) but are nearly absent in ECON, which instead contains many Numeric conflicts (71). Despite such diversity, DocChecker attains 80.1% accuracy on ECON, evidencing strong robustness.

(b) Performance and generalization. (i) DocChecker generalizes well across domains, achieving high F_1 on sentence-level NLI (SNLI: 93.1%, RTE-3: 91.1%). (ii) It remains robust on document-level datasets (ContraDoc: 91.4%, DOCNLI: 91.9%), showing effective long-range modeling. (iii) Evidence Hit Rate (EHR) is consistently high (average 85.7%), including CaseHOLD (86.4%) and StorySumm (86.2%), confirming accurate graph-based evidence retrieval.

(c) Sensitivity to graph structure. DocChecker's accuracy is influenced by how contradictions manifest structurally. Performance lags on vertex-level conflict datasets (*e.g.,* Knowledge Conflict: 80.3%) than those with more edge-level ones (*e.g.,* ContraDoc: 91.4%). This indicates that contradiction localization is highly dataset-dependent and underscores the need for explicit graph structure modeling to capture both attribute-level and relational conflicts.

(2) Sensitivity analysis. We examine how contradiction position, type, and domain affect DocChecker and GPT-4o on ContraDoc.

(a) Contradiction positions. Table 3 tests performance across three contextual spans: local (intra-paragraph), intra-document (inter-paragraph), and inter-document. (i) DocChecker performs strongly on local contradictions (80.82% Acc, 83.56% MR), beating GPT-4o (68.09% Acc). (ii) Accuracy drops to 76.81% for intra-document cases, reflecting the difficulty of detecting subtle cross-paragraph inconsistencies within shared context. (iii) Accuracy rises to 86.96% for inter-document contradictions, showing that graph alignment effectively captures structural divergences across sources. Across all spans, DocChecker consistently outperforms GPT-4o, whose accuracy degrades sharply on long-range contexts (down to 51.89%).

(b) Contradiction types. Table 3 examines detection difficulty across the seven contradiction types. (i) DocChecker performs best on structurally explicit categories, achieving 85.00% (Relational) and 83.74% (Causal), where RICO rules effectively capture clear logical dependencies. (ii) Performance is lower on semantically subtle or sequential types such as Negation (68.00%) and Temporal (72.86%), due to reliance on fine-grained semantic scope [92] and temporal reasoning [96], which are harder to generalize from structural patterns alone. (iii) Although GPT-4o shows high accuracy on Negation and Temporal (> 93%), its extremely low F_1 scores (5.00%, 52.17%) indicate severe false positives, whereas DocChecker remains balanced and reliable ($F_1 \geq 80.95\%$ for all types).

(c) Domain-specific analysis. Table 3 shows that DocChecker performs most stably on Stories (84.95% Acc.) and News (84.40%), with a high MR in Stories (95.70%), indicating reliable conflict localization. Performance is relatively lower on Encyclopedia text (74.47%) due to dense factual cross-referencing. This said, DocChecker remains consistently more robust than LLM baselines across all domains.

(3) DSCD tasks. We evaluate DocChecker against LLMs and pre-trained models across three core DSCD subtasks.

(a) Binary classification. This task evaluates whether a method is able to distinguish contradictory (positive) from non-contradictory (negative) texts. As shown in Task 1 of Table 4, DocChecker achieves near-perfect accuracy, reaching $P = 1.000$ and $F_1 = 0.992$ on ContraDoc, far surpassing GPT-5.2 ($F_1 = 0.727$) and Longformer ($F_1 = 0.520$). On ECON, although the performance gap narrows (*e.g.,* $F_1 = 0.859$ by Claude-3-7), DocChecker remains superior with $F_1 = 0.965$. These results indicate that DocChecker's graph-based modeling captures long-range dependencies more effectively than Transformer architectures and remains robust across domains.

(b) Top-3 contradiction evidence. Evaluated on CONTRADOC-POS, this task measures a model's ability to localize salient evidence sentences. As shown in Task 2 of Table 4, all models achieve $P = 1.000$ due to the positive-only setting, so Recall and Accuracy indicate evidence retrieval success. DocChecker attains the best results with

Dimension	Category	DocChecker		GPT-4o	
		Acc (%)	MR/F_1 (%)	Acc (%)	MR/F_1 (%)
Position (task 1)	Local	80.82	83.56	68.09	21.28
	Intra-document	76.81	89.04	51.89	17.92
	Inter-document	86.96	85.51	65.06	20.48
Type (task 2)	Factual	80.30	89.08	75.50	50.99
	Content	80.22	89.03	84.93	30.53
	Causal	83.74	91.15	67.88	18.49
	Negation	68.00	80.95	93.71	5.00
	Relational	85.00	91.89	91.89	3.92
	Numeric	75.25	85.87	94.04	52.63
	Temporal	72.86	84.30	94.54	52.17
Domain (task 3)	News	84.40	90.83	62.39	20.18
	Stories	84.95	95.70	72.04	30.11
	Encyclopedia	74.47	80.85	56.38	22.34

Table 3: Sensitivity analysis of DocChecker **vs. LLMs**

$F_1 = 0.962$ and Acc = 0.927, showing that graph fusion effectively highlights contradictory nodes. In contrast, GPT-5.2 and Claude-3-7 yield $F_1 = 0.856$ and $F_1 = 0.720$, frequently missing key evidence, while Gemini-3-Pro drops sharply ($F_1 = 0.390$); thus strong binary judgment does not necessarily imply reliable evidence localization.

(c) Evidence identification after binary judgment. This task tests whether a method can correctly locate contradictory evidence given a positive prediction. In Task 3 of Table 4, DocChecker achieves $J(D) \wedge V(E) = 0.914$, far surpassing GPT-5.2 (0.543) and Claude-3-7 (0.420). While LLMs maintain moderate $J(D)$ (0.56–0.72), they rarely provide verifiable evidence. In contrast, DocChecker attains high EHR (0.942) with balanced Precision/Recall (0.900/0.928), showing accurate localization. This verifies that explicit graph reasoning enables reliable DSCD with verifiable evidence.

(4) Document-level NLI. This task assesses ability to reason over long documents by classifying document pairs into *entailment*, *neutrality*, or *contradiction* (Table 5). DocChecker performs strongly in complex legal contexts, achieving 86.8% Acc and 0.929 F_1 on CaseHOLD, and 87.8% Acc with 0.935 F_1 on ContractNLI. Notably, DocChecker attains perfect Recall (1.000) on ContractNLI and surpasses Longformer by nearly 22% in F_1. Its consistent advantage over specialized models (*e.g.,* DeBERTa-v3-base, BART-base) highlights the effectiveness of RICO-based reasoning: by explicitly modeling document structure and integrating dispersed evidence through a unified graph, DocChecker captures long-range logical dependencies that standard neural attention often misses.

The results of sentence-level NLI tasks are consistent (see [1]).

Exp-2: Scalability. We assess the scalability of Learner, Detector, and DocChecker by varying: (1) document length $|D|$, (2) entity count $|\mathcal{E}|$, (3) dataset size $|\mathcal{D}|$, and (4) processor number $|n|$.

(1) Varying document length $|D|$. We first evaluate the scalability of the three as $|D|$ grows from 1,000 to 5,000 tokens on CaseHOLD and ConTRoL. As shown in Figures 4(a)–(b): (a) Learner takes 140.19-212.79 s on CaseHOLD and 122.74-191.87 s on ConTRoL. Its cost is driven mainly by the number and structural complexity of extracted dual-star patterns, whose density grows with document length. Specifically, Learner extracts an average of 528 rules (range: 487–577) from CaseHOLD and 896 rules (range: 862–943) from ConTRoL. (b) Detector exhibits minimal fluctuation, from 51.98-102.57 s on CaseHOLD and 44.87-160.44 s on ConTRoL, since reasoning runs on dual-star graphs with direct span mapping via f_{index}, yielding stable per-document latency. (c) End-to-end DocChecker costs 237.94-482.53 s on CaseHOLD and 255.19-549.18 s on ConTRoL, including graph construction, fusion, rule discovery, detection, and marking, indicating high efficiency. Even at 5,000 tokens, graph construction and join consume only 21.04% (101.54 s) on CaseHOLD and 16.76% (92.07 s) on ConTRoL, showing that graph abstraction effectively decouples reasoning cost.

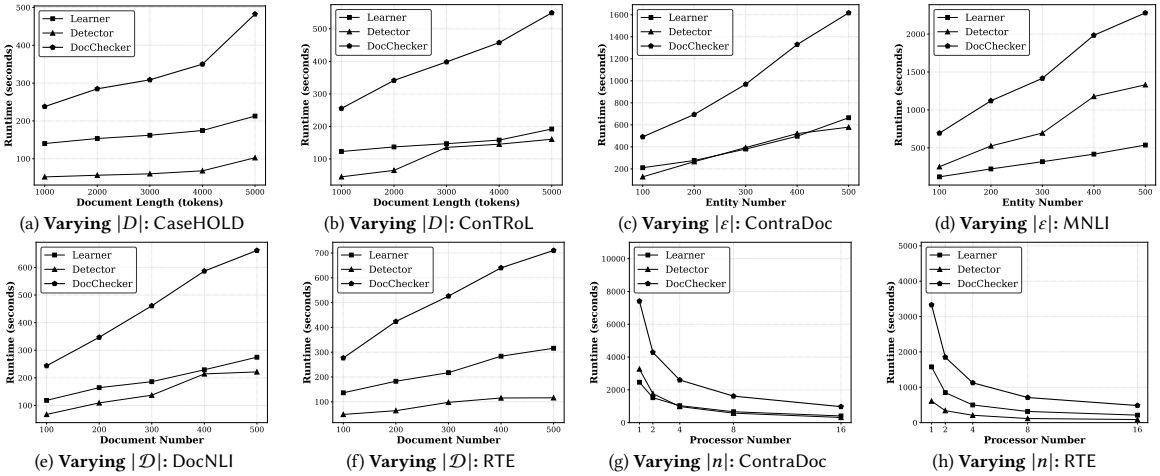
Model	Prec.	Rec.	F1	Acc.
Task 1: Binary Classification (CONTRADOC / ECON)				
DocChecker	1.000 / 0.960	0.985 / 0.970	0.992 / 0.965	0.986 / 0.953
GPT-5.2	0.983 / 0.941	0.577 / 0.480	0.727 / 0.636	0.608 / 0.633
Claude-3-7	0.977 / 0.941	0.433 / 0.790	0.600 / 0.859	0.477 / 0.827
Llama-3-70B	0.971 / 0.971	0.340 / 0.330	0.504 / 0.493	0.393 / 0.547
Gemini-3-Pro	1.000 / 0.930	0.400 / 0.400	0.571 / 0.559	0.640 / 0.580
Longformer	0.650 / 0.353	0.433 / 0.060	0.520 / 0.103	0.520 / 0.300
Task 2: Evidence Detection (CONTRADOC-POS)				
DocChecker	1.000	0.927	0.962	0.927
GPT-5.2	1.000	0.747	0.856	0.747
Claude-3-7	1.000	0.562	0.720	0.562
Llama-3-70B	1.000	0.412	0.584	0.412
Gemini-3-Pro	1.000	0.242	0.390	0.242
RoBERTa-mnli	1.000	0.322	0.413	0.322
Task 3: Joint Evidence Identification (CONTRADOC)				
Model	$J(D)$	$J \wedge V$	**EHR**	**P**	**R**
DocChecker	0.992	0.914	0.942	0.900	0.928
GPT-5.2	0.719	0.543	0.957	0.537	0.550
Claude-3-7	0.682	0.420	0.586	0.415	0.425
Qwen-Max	0.561	0.361	0.813	0.406	0.325
Gemini-3-Pro	0.625	0.400	0.950	0.346	0.475
DeepSeek-V3	0.644	0.356	0.684	0.394	0.325

Table 4: Performance evaluation of three DSCD subtasks

(2) Varying entity number $|\mathcal{E}|$. Figures 4(c)–(d) evaluate runtime sensitivity as graph size (driven by $|\mathcal{E}|$) scales from 100 to 500 nodes on ContraDoc and MNLI. (a) Learner grows from 210.74–664.56 s on ContraDoc and 117.72–536.85 s on MNLI; BFS-based pattern extraction grows with denser dual-star structures, while LLM pruning remains stable. (b) Detector shifts from 127.86–579.24 s on ContraDoc and 250.94-1331.25 s on MNLI; even at 500 nodes, dual-star topology keeps matching localized, keeping per-contradiction latency within 278.48 ms and 594.31 ms, respectively. (c) Overall DocChecker rises from 491.11–1616.97 s on ContraDoc and 693.93–2279.40 s on MNLI. As 78.70% of real-world dual-star patterns involve < 200 entities, practical runtime remains around 692 s on average even at $|\mathcal{E}| = 500$, confirming no exponential blowup.

(3) Varying dataset $|\mathcal{D}|$. Figures 4(e)–(f) evaluate scalability as the corpus grows from 100 to 500 documents on the DocNLI and RTE datasets. (a) Learner increases moderately from 118.34s to 274.69s on DocNLI and 136.21s to 315.94s on RTE, as iterative pruning limits redundant patterns and LLM calls. (b) Detector maintains stable per-document latency (0.443s on DocNLI and 0.232s on RTE), with total runtime bounded to 221.46s (DocNLI) and 116.01s (RTE) on 16 processors, showing reasoning is localized to dual-star subgraphs rather than corpus size. (c) DocChecker rises from 243.59s to 661.61s on DocNLI and 276.63s to 710.23s on RTE, confirming that offline discovery scales with input volume while online reasoning remains effectively size-invariant for large-scale monitoring.

(4) Varying processor number $|n|$. Figures 4(g)–(h) test scaling as n grows from 1 to 16 $(1, 2, 4, 8, 16)$ on the ContraDoc and RTE datasets. (a) Learner improves nearly linearly, falling from 2465.38s to 407.82 s on ContraDoc and 1576.86s to 212.19 s on RTE as parallel workers mask LLM latency. (b) Detector keeps stable per-document time (0.403s), while corpus-level runtime drops from 3270.26s to



(a) **Varying** $|D|$: CaseHOLD (b) **Varying** $|D|$: ConTRoL (c) **Varying** $|\varepsilon|$: ContraDoc (d) **Varying** $|\varepsilon|$: MNLI

(e) **Varying** $|\mathcal{D}|$: DocNLI (f) **Varying** $|\mathcal{D}|$: RTE (g) **Varying** $|n|$: ContraDoc (h) **Varying** $|n|$: RTE

Figure 4: Scalability trends across key variables

Model	CaseHOLD				ContractNLI			
	Acc	P	R	F1	Acc	P	R	F1
DocChecker	0.868	0.879	0.986	0.929	0.878	0.878	1.000	0.935
Longformer-base-4096	0.310	0.347	0.310	0.321	0.712	0.734	0.712	0.716
DeBERTa-v3-base	0.676	0.677	0.676	0.676	0.704	0.733	0.704	0.702
BART-base	0.672	0.680	0.672	0.673	0.508	0.451	0.508	0.477
DistilBERT-base	0.605	0.608	0.605	0.606	0.684	0.699	0.684	0.680

Table 5: Performance on legal QA and document-level NLI

Model Configuration	Accuracy (%)	F1-score (%)	Runtime (s)
DocChecker (Full Model)	85.2	92.0	551.3
Ablations			
w/o Graph Join	63.0	77.3	481.1
w/o Graph Enrichment	73.8	85.0	445.7
w/o Meta-learning	41.0	58.1	over 5 hours

Table 6: Ablation study of key DocChecker **components.**

315.33 s on ContraDoc and 609.12s to 87.54 s on RTE in parallel. (c) DocChecker time reduces from 7410.11s to 974.44s on ContraDoc and 3330.40s to 483.56s on RTE, verifying parallel scalability.

Exp-3: Ablation study. We quantify the contributions of key DocChecker components to effectiveness (Accuracy) and efficiency (Runtime) via three ablations on StorySumm: (a) *Graph join*: Full DocChecker (with cross-document linking) versus a variant without graph aggregation. (b) *Graph enrichment*: Full DocChecker (with external KG enrichment) versus a version without enrichment. (c) *Discovery metrics*: Learning-based metrics versus traditional *Support*. The support threshold is set to 3, with a 5-hour timeout; Learner stops at timeout; all mined rules are returned.

Table 6 shows the the contributions of each component.

(1) Graph components. Removing Graph Join causes the largest drop, reducing Accuracy by 22.2 points to 63.0%, confirming cross-document linking as the most critical factor for high-quality contradiction detection. Disabling Graph Enrichment also degrades accuracy (73.8%), showing that external knowledge provides useful context, although its effect is secondary to structural linking.

(2) Meta learning. Restoring support-based metrics has severe effect: accuracy drops to 41.0% and runtime exceeds 5 hours. Thus meta learning is crucial not only for accuracy but also for computational feasibility, enabling efficient generalization and convergence.

(3) Efficiency and scalability. Despite added structural modeling, the Full Model remains efficient (551.3 s), beating ablated variants by a large margin. This shows that DocChecker achieves a strong accuracy–efficiency balance and scales well to dense text analysis.

Summary. We find the following. (1) *Effectiveness.* DocChecker achieves SOTA accuracy across 15 datasets, reaching 93.1% F_1 on SNLI and 91.4% on ContraDoc. It excels on document-level NLI, reaching 87.8% *Acc* on ContractNLI, significantly outperforming specialized models (*e.g.,* DeBERTa-v3), particularly in complex legal contexts. (2) *Robustness.* DocChecker is resilient to long-range dependencies, sustaining 76.81% (intra-) and 86.96% (inter-document) accuracy, surpassing GPT-4o's 51.89% and 65.06% on ContraDoc. In DSCD tasks, it attains near-perfect binary accuracy (0.992 F_1) and superior evidence localization (0.962 F_1), providing more reliable, verifiable evidence than Claude-3-7. (3) *Efficiency and Scalability.* DocChecker scales linearly with document length (1k–5k tokens) and entity size (100–500 nodes). Its parallel design is highly efficient, reducing ContraDoc runtime from 7410.11 s to 974.44 s (from 1 to 16 processors) via localized dual-star reasoning. (4) *Ablation Findings.* Graph Join is the most critical component, yielding a 22.2% accuracy gain. Meta-learning is essential for feasibility: removing it collapses accuracy to 41.0% and leads to runtime timeouts.

8 CONCLUSION

The work proposes a logical approach to detecting textual contradictions, with the following novelties: (a) RICOs, a class of rules that capture both entity- and relation-level inconsistencies while remaining tractable (PTIME); (b) DocChecker, a system that integrates graph conversion, enrichment, fusion, and analytics to model and reason document contradictions; (c) Learner, a learning-based rule discovery algorithm with new effectiveness metrics beyond traditional support; and (d) Detector, a parallelly scalable algorithm that uniformly detects intra- and inter-document contradictions. Our experimental study has verified that DocChecker is practical.

Future work includes (a) extending DocChecker to *cross-modal contradiction detection* (*e.g.,* text–table, text–image) by developing multimodal RICOs while retaining tractability, and (b) enhancing *robustness to LLM noise and bias* via cross-model verification, confidence calibration, and formal consistency checks to mitigate hallucinations and ensure trustworthy rule discovery and detection.

REFERENCES

[1] 2026. Code, datasets and full version. https://anonymous.4open.science/r/Doc4LLM-CE2E/.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, and Shyamal Anadkat. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). https://arxiv.org/abs/2303.08774

[4] João Paulo Aires and Felipe Meneguzzi. 2017. Norm Conflict Identification Using Deep Learning. In *AAMAS Workshops*. 194–207.

[5] Abdulaziz Alamri. 2016. *The Detection of Contradictory Claims in Biomedical Abstracts*. Ph. D. Dissertation. University of Sheffield.

[6] Abdulaziz Alamri and Mark Stevenson. 2016. A Corpus of Potentially Contradictory Research Claims from Cardiovascular Research. *Journal of Biomedical Semantics* 7 (2016), 36.

[7] Alan Ansell, Felipe Bravo-Marquez, and Bernhard Pfahringer. 2021. PolyLM: Learning about Polysemy through Language Modeling. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 563–574.

[8] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf Technical Report.

[9] Marcelo Arenas, Leopoldo Bertossi, and Jan Chomicki. 1999. Consistent Query Answers in Inconsistent Databases. In *PODS*. 68–79.

[10] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, and Fei Huang. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023). https://arxiv.org/abs/2309.16609

[11] Xianchun Bao, Zian Bao, Qingsong Duan, Wenfei Fan, Hui Lei, Daji Li, Wei Lin, Peng Liu, Zhicong Lv, Mingliang Ouyang, Jiale Peng, Jing Zhang, Runxiao Zhao, Shuai Tang, Shuping Zhou, Yaoshu Wang, Qiyuan Wei, Min Xie, Jing Zhang, Xin Zhang, Runxiao Zhao, and Shuping Zhou. 2024. Rock: Cleaning Data by Embedding ML in Logic Rules. In *SIGMOD (industrial track)*.

[12] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150* (2020). https://arxiv.org/abs/2004.05150

[13] Christopher M. Bishop and Nasser M. Nasrabadi. 2006. *Pattern Recognition and Machine Learning*. Springer.

[14] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A Large Annotated Corpus for Learning Natural Language Inference. In *EMNLP*. 632–642.

[15] Hung-Ting Chen, Michael Zhang, and Eunsol Choi. 2022. Rich Knowledge Sources Bring Complex Knowledge Conflicts: Recalibrating Models to Reflect Conflicting Evidence. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2292–2307.

[16] Yuhao Chen, Yuanjie Lyu, Shuochen Liu, Chao Zhang, Junhui Lv, and Tong Xu. 2025. Think Wider, Detect Sharper: Reinforced Reference Coverage for Document-Level Self-Contradiction Detection. In *EMNLP*. 1273–1288.

[17] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, and Jacob Devlin. 2024. Scaling Instruction-Finetuned Language Models. *JMLR* 25, 70 (2024), 1–53.

[18] Peter Coates and Frank Breitinger. 2023. Identifying document similarity using a fast estimation of the Levenshtein Distance based on compression and signatures. *arXiv preprint arXiv:2307.11496* (2023). https://arxiv.org/abs/2307.11496

[19] Mark Coddington and Logan Molyneux. 2023. When sources contradict: The epistemological functions of contradiction in news texts. *Journalism Studies* 24, 10 (2023), 1316–1333.

[20] Cleo Condoravdi, Dick Crouch, Valeria De Paiva, Reinhard Stolle, and Daniel Bobrow. 2003. Entailment, Intensionality and Text Understanding. In *NAACL*. 38–45.

[21] Gao Cong, Wenfei Fan, Floris Geerts, Xibei Jia, and Shuai Ma. 2007. Improving Data Quality: Consistency and Accuracy. In *VLDB*. 315–326.

[22] Ido Dagan, Oren Glickman, and Bernardo Magnini. 2005. The PASCAL Recognising Textual Entailment Challenge. In *Machine Learning Challenges Workshop*. Springer, 177–190.

[23] Marie-Catherine de Marneffe, Anna N. Rafferty, and Christopher D. Manning. 2008. Finding Contradictions in Text. In *ACL*. The Association for Computer Linguistics, 1039–1047.

[24] Tobias Deußer, Maren Pielka, Lisa Pucknat, Basil Jacob, Tim Dilmaghani, Mahdis Nourimand, Bernd Kliem, Rüdiger Loitz, Christian Bauckhage, and Rafet Sifa. 2023. Contradiction Detection in Financial Reports. In *NLDL*.

[25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *NAACL*. 4171–4186.

[26] Xin Dong, Evgeniy Gabrilovich, Geremy Heitz, Wilko Horn, Ni Lao, Kevin Murphy, Thomas Strohmann, Shaohua Sun, and Wei Zhang. 2014. Knowledge Vault: A Web-Scale Approach to Probabilistic Knowledge Fusion. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 601–610.

[27] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models Through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning*.

[28] David Easley, Jon Kleinberg, et al. 2010. *Networks, Crowds, and Markets: Reasoning about a Highly Connected World*. Vol. 1. Cambridge university press Cambridge.

[29] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024). https://arxiv.org/abs/2404.16130

[30] Wenfei Fan, Wenzhi Fu, Ruochun Jin, Ping Lu, and Chao Tian. 2022. Discovering Association Rules from Big Graphs. *PVLDB* 15, 7 (2022), 1479–1492.

[31] Wenfei Fan, Wenzhi Fu, Ruochin Jin, Ping Lu, and Chao Tian. 2023. Making It Tractable to Catch Duplicates and Conflicts in Graphs. *SIGMOD* 1, 1 (2023), 86:1–86:28.

[32] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *TODS* 33, 2 (2008), 6:1–6:48.

[33] Wenfei Fan, Floris Geerts, Nan Tang, and Wenyuan Yu. 2014. Conflict resolution with data currency and consistency. *J. Data and Information Quality* 5, 1-2 (2014), 6:1–6:37.

[34] Wenfei Fan, Ruochun Jin, Muyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing Associations in Graphs. *PVLDB* 13, 11 (2020), 1863–1876.

[35] Wenfei Fan, Muyang Liu, Shuhao Liu, and Chao Tian. 2024. Capturing More Associations by Referencing External Graphs. *PVLDB* 17, 6 (2024), 1173–1186.

[36] Wenfei Fan, Muyang Liu, Ping Lu, and Chao Tian. 2023. Making It Tractable to Catch Duplicates and Conflicts in Graphs. *ACM Trans. on Database Systems* (2023).

[37] Wenfei Fan, Xueli Liu, Ping Lu, and Chao Tian. 2020. Catching Numeric Inconsistencies in Graphs. *TODS* 45, 2 (2020), 1–47.

[38] Wenfei Fan and Ping Lu. 2019. Dependencies for Graphs. *ACM TODS* 44, 2 (2019), 1–40.

[39] Wenfei Fan, Ping Lu, and Chao Tian. 2020. Unifying Logic Rules and Machine Learning for Entity Enhancing. *Sci. China Inf. Sci.* 63, 7 (2020).

[40] Wenfei Fan, Ping Lu, Chao Tian, and Jingren Zhou. 2019. Deducing Certain Fixes to Graphs. *PVLDB* 12, 7 (2019), 752–765.

[41] Wenfei Fan, Chao Tian, Yanghao Wang, and Qiang Yin. 2021. Parallel Discrepancy Detection and Incremental Detection. *PVLDB* 14, 8 (2021), 1351–1364.

[42] Wenfei Fan, Yinghui Wu, and Jingbo Xu. 2016. Functional dependencies for graphs. In *SIGMOD*.

[43] Wenfei Fan, Jingbo Xu, Yinghui Wu, Wenyuan Yu, Jiaxin Jiang, Bohan Zhang, Zeyu Zheng, Yang Cao, and Chao Tian. 2017. Parallelizing Sequential Graph Computations. In *SIGMOD*.

[44] Anna Fariha, Ashish Tiwari, Alexandra Meliou, Arjun Radhakrishna, and Sumit Gulwani. 2021. CoCo: Interactive Exploration of Conformance Constraints for Data Understanding and Data Cleaning. In *SIGMOD*. ACM, 2706–2710.

[45] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern recognition letters* 27, 8 (2006), 861–874.

[46] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and Robust Automated Machine Learning. In *NeurIPS*. 2962–2970.

[47] Lampros Flokas, Weiyuan Wu, Yejia Liu, Jiannan Wang, Nakul Verma, and Eugene Wu. 2022. Complaint-Driven Training Data Debugging at Interactive Speeds. In *SIGMOD*. 369–383.

[48] Etienne Fortier-Dubois and Domenic Rosati. 2023. Using Contradictions Improves Question Answering Systems. In *ACL*. 827–840.

[49] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.

[50] Gemini Team and Google. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805* (2023). https://arxiv.org/abs/2312.11805

[51] Danilo Giampiccolo, Bernardo Magnini, Ido Dagan, and William B. Dolan. 2007. The Third PASCAL Recognizing Textual Entailment Challenge. In *ACL*. 1–9.

[52] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *ICML*. 1263–1272.

[53] Joseph E. Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: Distributed Graph-Parallel Computation on Natural Graphs. In *OSDI*. 17–30.

[54] Rihan Hai, Christos Koutras, Andra Ionescu, Ziyu Li, Wenbo Sun, Jessie van Schijndel, Yan Kang, and Asterios Katsifodimos. 2023. Amalur: Data Integration Meets Machine Learning. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 3729–3739.

[55] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NeurIPS*, Vol. 30. 1025–1035.

[56] Sanda Harabagiu, Andrew Hickl, and Finley Lacatusu. 2006. Negation, contrast and contradiction in text processing. In *AAAI*, Vol. 6. 755–762.

[57] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543* (2021). https://arxiv.org/abs/2111.09543

[58] Alireza Heidari, Joshua McGrath, Ihab F Ilyas, and Theodoros Rekatsinas. 2019. HoloDetect: Few-Shot Learning for Error Detection. In *SIGMOD*. 829–846.

[59] Andrew Hickl, John Williams, Jeremy Bensley, Kirk Roberts, Bryan Rink, and Ying Shi. 2006. Recognizing Textual Entailment with LCC's GROUNDHOG System. In *Proceedings of the Second PASCAL Challenges Workshop*, Vol. 18. 1–4.

[60] Benjamin Hilprecht, Christian Hammacher, Eduardo Souza dos Reis, Mohamed Abdelaal, and Carsten Binnig. 2023. DiffML: End-to-end Differentiable ML Pipelines. In *DEEM@SIGMOD*. ACM, 7:1–7:7.

[61] Johannes Hoffart, Mohamed Amir Yosef, Ilaria Bordino, Hagen Fürstenau, Manfred Pinkal, Marc Spaniol, Bilyana Taneva, Stefan Thater, and Gerhard Weikum. 2011. Robust Disambiguation of Named Entities in Text. In *EMNLP*. 782–792.

[62] Cheng Hsu, Cheng-Te Li, Diego Saez-Trumper, and Yi-Zhan Hsu. 2021. Wikicontradiction: Detecting Self-Contradiction Articles on Wikipedia. In *BigData*. 427–436.

[63] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2020. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations*. https://openreview.net/forum?id=HJlWWJSFDH

[64] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous Graph Transformer. In *Proceedings of the Web Conference 2020*. 2704–2710.

[65] Pengcheng Jiang, Lang Cao, Ruike Zhu, Minhao Jiang, Yunyi Zhang, Jimeng Sun, and Jiawei Han. 2025. RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation. *CoRR* abs/2502.10996 (2025). https://doi.org/10.48550/arXiv.2502.10996

[66] Cheng Jiayang, Chunkit Chan, Qianqian Zhuang, Lin Qiu, Tianhang Zhang, Tengxiao Liu, Yangqiu Song, Yue Zhang, Pengfei Liu, and Zheng Zhang. 2024. ECON: On the Detection and Resolution of Evidence Conflicts. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 7816–7844.

[67] Di Jin, Sijia Liu, Yang Liu, and Dilek Hakkani-Tur. 2022. Improving Bot Response Contradiction Detection via Utterance Rewriting. In *SIGDIAL*. 605–614.

[68] Bojan Karlas, Peng Li, Renzhi Wu, Nezihe Merve Gürel, Xu Chu, Wentao Wu, and Ce Zhang. 2020. Nearest Neighbor Classifiers over Incomplete Information: From Certain Answers to Certain Predictions. *CoRR* abs/2005.05117 (2020). ArXiv:2005.05117 https://arxiv.org/abs/2005.05117

[69] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.

[70] Yuta Koreeda and Christopher D. Manning. 2021. ContractNLI: A Dataset for Document-Level Natural Language Inference for Contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 1907–1919.

[71] Sanjay Krishnan, Michael J. Franklin, Ken Goldberg, and Eugene Wu. 2017. BoostClean: Automated Error Detection and Repair for Machine Learning. *CoRR* abs/1711.01299 (2017).

[72] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J. Franklin, and Ken Goldberg. 2016. ActiveClean: Interactive Data Cleaning For Statistical Modeling. *PVLDB* 9, 12 (2016), 948–959.

[73] Sanjay Krishnan and Eugene Wu. 2019. AlphaClean: Automatic Generation of Data Cleaning Pipelines. *CoRR* abs/1904.11827 (2019). http://arxiv.org/abs/1904.11827

[74] Clyde P. Kruskal, Larry Rudolph, and Marc Snir. 1990. A Complexity Theory of Efficient Parallel Algorithms. *Theoretical Computer Science* 71, 1 (1990), 95–132.

[75] Piroska Lendvai, Isabelle Augenstein, Kalina Bontcheva, and Thierry Declerck. 2016. Monolingual Social Media Datasets for Detecting Contradiction and Entailment. In *LREC*. 4602–4605.

[76] Piroska Lendvai and Uwe Reichel. 2016. Contradiction Detection for Rumorous Claims. In *Proceedings of the Workshop on Extra-Propositional Aspects of Meaning in Computational Linguistics (ExProM)*. 31–40.

[77] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-Training for Natural Language Generation, Translation, and Comprehension. In *ACL*. 7871–7880.

[78] Chuqin Li, Xi Niu, Ahmad Al-Doulat, and Noseong Park. 2018. A Computational Approach to Finding Contradictions in User Opinionated Text. In *ASONAM*. 351–356.

[79] Jierui Li, Vipul Raheja, and Dhruv Kumar. 2024. ContraDoc: Understanding Self-Contradictions in Documents with Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 6509–6523.

[80] Peng Li, Zhiyi Chen, Xu Chu, and Kexin Rong. 2023. DiffPrep: Differentiable Data Preprocessing Pipeline Search for Learning over Tabular Data. *SIGMOD* 1, 2 (2023), 183:1–183:26.

[81] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. In *ICDE*. IEEE, 13–24.

[82] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, and Chong Ruan. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024). https://arxiv.org/abs/2412.19437

[83] Hanmeng Liu, Leyang Cui, Jian Liu, and Yue Zhang. 2021. Natural language inference in context-investigating contextual reasoning over long texts. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 13388–13396.

[84] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019). https://arxiv.org/abs/1907.11692

[85] Zifan Liu, Zhechun Zhou, and Theodoros Rekatsinas. 2020. Picket: Self-supervised Data Diagnostics for ML Pipelines. *CoRR* abs/2006.04730v1 (2020). https://arxiv.org/abs/2006.04730v1

[86] Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-Based Knowledge Conflicts in Question Answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 7052–7063.

[87] Bill MacCartney, Trond Grenager, Marie-Catherine de Marneffe, Daniel Cer, and Christopher D. Manning. 2006. Learning to Recognize Features of Valid Textual Entailments. In *ACL*. 41–48.

[88] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *SIGMOD*. 865–882.

[89] Stanislav Malyshev, Markus Krötzsch, Larry González, Julius Gonsior, and Adrian Bielefeldt. 2018. Getting the Most Out of Wikidata: Semantic Technology Usage in Wikipedia's Knowledge Graph. In *ISWC*. 376–394.

[90] Puneet Mathur, Gautam Kunapuli, Riyaz Bhat, Manish Shrivastava, Dinesh Manocha, and Maneesh Singh. 2022. DocInfer: Document-level natural language inference using optimal evidence selection. In *EMNLP*. 809–824.

[91] Christian Meilicke, Melisachew Wudage Chekol, Manuel Fink, and Heiner Stuckenschmidt. 2020. Reinforced Anytime Bottom-Up Rule Learning for Knowledge Graph Completion. *arXiv preprint arXiv:2004.04412* (2020). https://arxiv.org/abs/2004.04412

[92] Roser Morante and Caroline Sporleder. 2012. Modality and Negation: An Introduction to the Special Issue. *Comput. Linguist.* 38, 2 (2012), 223–260.

[93] Yixin Nie, Mary Williamson, Mohit Bansal, Douwe Kiela, and Jason Weston. 2021. I Like Fish, Especially Dolphins: Addressing Contradictions in Dialogue Modeling. In *ACL*. 1699–1713.

[94] OpenAI. 2025. OpenAI GPT-5: A Comprehensive Technical Overview. https://cirra.ai/articles/gpt-5-technical-overview

[95] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *TKDE* 36, 7 (2024), 3580–3599.

[96] James Pustejovsky, José M. Castano, Robert Ingria, Roser Sauri, Robert J. Gaizauskas, Andrea Setzer, Graham Katz, and Dragomir R. Radev. 2003. TimeML: Robust Specification of Event and Temporal Expressions in Text. *New Directions in Question Answering* 3 (2003), 28–34.

[97] Qiu Ran, Yankai Lin, Peng Li, Jie Zhou, and Zhiyuan Liu. 2019. NumNet: Machine Reading Comprehension with Numerical Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2474–2484.

[98] Alexander Ratner, Stephen H. Bach, Henry R. Ehrenberg, Jason Alan Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *PVLDB* 11, 3 (2017), 269–282.

[99] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*. 3980–3990.

[100] Graciela Rosemblat, Marcelo Fiszman, Dongwook Shin, and Halil Kilicoglu. 2019. Towards a Characterization of Apparent Contradictions in the Biomedical Literature Using Context Analysis. *JBI* 98 (2019), 103275.

[101] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108* (2019). https://arxiv.org/abs/1910.01108

[102] Farzaneh Sarafraz. 2012. *Finding Conflicting Statements in the Biomedical Literature*. Ph. D. Dissertation. The University of Manchester.

[103] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *ESWC*. 593–607.

[104] Tal Schuster, Sihao Chen, Senaka Buthpitiya, Alex Fabrikant, and Donald Metzler. 2022. Stretching Sentence-pair NLI Models to Reason over Long Documents and Clusters. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 394–412.

[105] Özge Sevgili, Artem Shelmanov, Mikhail Arkhipov, Alexander Panchenko, and

Chris Biemann. 2022. Neural Entity Linking: A Survey of Models Based on Deep Learning. *Semantic Web* 13, 3 (2022), 527–570.
[106] Wei Shen, Jianyong Wang, and Jiawei Han. 2014. Entity Linking with a Knowledge Base: Issues, Techniques, and Solutions. *TKDE* 27, 2 (2014), 443–460.
[107] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS* 36 (2023), 8634–8652.
[108] Zhaochen Su, Zecheng Tang, Xinyan Guan, Lijun Wu, Min Zhang, and Juntao Li. 2022. Improving Temporal Generalization of Pre-Trained Language Models with Lexical Semantic Change. In *EMNLP*. 6380–6393.
[109] Zhaochen Su, Jun Zhang, Xiaoye Qu, Tong Zhu, Yanshu Li, Jiashuo Sun, Juntao Li, Min Zhang, and Yu Cheng. 2024. ConflictBank: A Benchmark for Evaluating the Influence of Knowledge Conflicts in LLMs. In *NeurIPS*, Vol. 37. 103242–103268.
[110] Melanie Subbiah, Faisal Ladhak, Akankshya Mishra, Griffin Thomas Adams, Lydia Chilton, and Kathleen McKeown. 2024. StorySumm: Evaluating Faithfulness in Story Summarization. In *EMNLP*. 9988–10005.
[111] Katia P. Sycara. 1993. Machine learning for intelligent support of conflict resolution. *Decision Support Systems* 10, 2 (1993), 121–136.
[112] Ki Hyun Tae, Yuji Roh, Young Hun Oh, Hyunsu Kim, and Steven Euijong Whang. 2019. Data Cleaning for Accurate, Fair, and Robust Models: A Big Data - AI Integration Approach. In *DEEM@SIGMOD*. ACM, 5:1–5:4.
[113] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 809–819.
[114] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, and Shruti Bhosale. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783
[115] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Commun. ACM* 33, 8 (1990), 103–111.
[116] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. arXiv:1710.10903 [stat.ML] https://arxiv.org/abs/1710.10903
[117] Larysa Visengeriyeva and Ziawasch Abedjan. 2018. Metadata-driven error detection. In *SSDBM*. 1:1–1:12.
[118] Denny Vrandečić. 2012. Wikidata: A New Platform for Collaborative Data Collection. In *Proceedings of the 21st International Conference on World Wide Web*. 1063–1064.
[119] Alexander Wan, Eric Wallace, and Dan Klein. 2024. What Evidence Do Language Models Find Convincing? arXiv:2402.11782 [cs.CL] https://arxiv.org/abs/2402.11782
[120] Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. 2025. LLMs Know What They Need: Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation. In *COLING*. 2379–2400.
[121] Yike Wang, Shangbin Feng, Heng Wang, Weijia Shi, Vidhisha Balachandran, Tianxing He, and Yulia Tsvetkov. 2024. Resolving Knowledge Conflicts in Large Language Models. arXiv:2310.00935 [cs.CL] https://arxiv.org/abs/2310.00935
[122] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *ACL*. 13484–13508.
[123] Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding Through Inference. In *Proceedings of the 2018 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long papers)*. 1112–1122.
[124] Weiyuan Wu, Lampros Flokas, Eugene Wu, and Jiannan Wang. 2020. Complaint-driven Training Data Debugging for Query 2.0. In *SIGMOD*. ACM, 1317–1334.
[125] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2023. Adaptive Chameleon or Stubborn Sloth: Revealing the Behavior of Large Language Models in Knowledge Conflicts. In *ICLR*.
[126] Wenhan Xiong, Thien Hoang, and William Yang Wang. 2017. DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. 564–573.
[127] Haike Xu, Zongyu Lin, Yizhou Sun, Kai-Wei Chang, and Piotr Indyk. 2024. SparseCL: Sparse Contrastive Learning for Contradiction Retrieval. arXiv:2406.10746 [cs.CL] https://arxiv.org/abs/2406.10746
[128] Rongwu Xu, Brian Lin, Shujian Yang, Tianqi Zhang, Weiyan Shi, Tianwei Zhang, Zhixuan Fang, Wei Xu, and Han Qiu. 2024. The Earth Is Flat Because...: Investigating LLMs' Belief Towards Misinformation via Persuasive Conversation. In *ACL*. 16259–16303.
[129] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R. Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. *NeurIPS* 32 (2019).
[130] Wenpeng Yin, Dragomir Radev, and Caiming Xiong. 2021. DocNLI: A Large-Scale Dataset for Document-Level Natural Language Inference. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. 4913–4922.
[131] Zhangdie Yuan and Andreas Vlachos. 2023. Zero-Shot Fact-Checking with Semantic Triples and Knowledge Graphs. *CoRR* abs/2312.11785 (2023).
[132] Fabio Massimo Zanzotto, Marco Pennacchiotti, and Alessandro Moschitti. 2007. Shallow Semantic in Fast Textual Entailment Rule Learners. In *ACL*. 72–77.
[133] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. 2024. GLiNER: Generalist Model for Named Entity Recognition Using Bidirectional Transformer. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 5364–5376.
[134] Jiawei Zhang, Haopeng Zhang, Congying Xia, and Li Sun. 2020. Graph-Bert: Only Attention is Needed for Learning Graph Representations. (2020). arXiv:2001.05140 [cs.LG] https://arxiv.org/abs/2001.05140
[135] Chujie Zheng, Jinfeng Zhou, Yinhe Zheng, Libiao Peng, Zhen Guo, Wenquan Wu, Zheng-Yu Niu, Hua Wu, and Minlie Huang. 2022. CDConv: A Benchmark for Contradiction Detection in Chinese Conversations. In *EMNLP*. 18–29.
[136] Lucia Zheng, Neel Guha, Brandon R. Anderson, Peter Henderson, and Daniel E. Ho. 2021. When Does Pretraining Help? Assessing Self-Supervised Learning for Law and the CaseHOLD Dataset of 53,000+ Legal Holdings. In *Proceedings of the eighteenth international conference on artificial intelligence and law*. 159–168.

A PROOF OF THEOREM 1

Proof: Given a graph G, a document collection $\mathcal{D}$, and a set Σ of RICOs, validation asks whether $(G, \mathcal{D}) \models \Sigma$, *i.e.,* for every $\varphi = Q[x_0, y_0](X \rightarrow q_0) \in \Sigma$ there is no valuation h of φ such that $h \models X$, but $h \not\models q_0$. It suffices to validate a single φ in PTIME.

Predicate evaluation. Each predicate in RICO is polynomially checkable. Attribute predicates $z.A \oplus c$ and $x.A \oplus y.B$ require constant-time lookups and scalar comparison. Binding $S(x, s)$ performs indexed span lookup in at most linear time. Textual and ML predicates (T, $\mathcal{M}$, τ) have PTIME cost relative to $|G|$ and $|\mathcal{D}|$, following standard bounded-input assumptions in dependency validation [31, 38]. Thus, every atomic predicate in X and q_0 is decidable in PTIME.

Pattern matching and violation checking. Instead of enumerating all subgraphs of Q, pattern matching follows the PTIME strategy in [36]. For each rule $\varphi \in \Sigma$, the algorithm iterates over vertex pairs $(u, v) \in V_G \times V_G$ as candidates for (x_0, y_0). It first checks unary predicates on x_0 and y_0, reducing the search space to $O(|V_G|^2)$ root candidates. If satisfied, matching extends to leaf variables (x_i, y_j) and incident edges (e_{x_i}, e_{y_j}). RICO 's structural constraints then ensure tractability: by Restriction (1) of the definition of RICOs (Section 3.2), binary predicates appear only across the two stars; by Restriction (2), each leaf and its incident edge participate in *at most one* binary predicate. Consequently, verifying whether a valuation satisfies $X \wedge \neg q_0$ reduces to independent checks over neighbors of u and v, effectively a linear scan or hash-join over adjacency lists, rather than an exponential search over leaf combinations [36].

Complexity. Let $n = |V_G|$. For each rule $\varphi \in \Sigma$, the algorithm examines $O(n^2)$ candidate root pairs (u, v). For each pair, verifying the binary predicates requires scanning the neighbors of u and v. Since each leaf participates in at most one binary predicate, checking k binary predicates reduces to k independent neighbor checks, costing $O(k \cdot \deg(G)^2)$ in the worst case. Thus, the total complexity is

$$O(|\Sigma| \cdot n^2 \cdot \deg(G)^2).$$

Note that the exponential factor $\deg(G)^{|\bar{x}|+|\bar{y}|}$ that appears in generic graph pattern matching is avoided because leaf-pair valuations are decoupled. Hence, RICO validation is in PTIME. □

B CONTRADICTION DETECTION

We next provide a detailed explanation of the Contradiction Detection and Document Marking procedures of DocChecker. We focus on the chase process in contradiction detection, prove its Church-Rosser property, and present the parallel detection algorithm. We also describe how detected contradictions are mapped back to the original documents for document marking.

B.1 The chase process

Facts. To track facts for contradictions, for each attribute $x.A$ (resp. $e_x.A$), we maintain relations $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$), where $\otimes$ ranges over $=, \neq, <, \leq, >, \geq$. Each $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) consists of attributes $y.B$ (resp. $e_y.B$) or constants c such that $x.A \otimes y.B$ (resp. $e_x.A \otimes e_y.B$) and $x.A \otimes c$ (resp. $e_x.A \otimes c$) are either in the initial ground truth of Γ (see below) or generated during the chase.

In particular, if $x.A$ (resp. $e_x.A$) is in contradiction, then $\tau(x.A, y.B)$ (resp., $\tau(e_x.A, e_y.B)$) may include a contradiction type (*i.e.,* casual, content, etc.) as a special value for label.

We ensure that each $[x.A]_=$ (resp. $[e_x.A]_=$) forms an equivalence relation, *i.e.,* it is reflexive, symmetric, and transitive, while each $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) for any other $\otimes$ is reflexive and transitive. Moreover, each $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) is *valid, i.e.,* no two constants within the same class conflict; for example, $[x.A]_=$ (resp. $[e_x.A]_=$) cannot contain constants c_1 and c_2 with $c_1 \neq c_2$.

Ground truth. The initial fact set, denoted Γ, is collected and validated by users, domain experts or crowd-sourcing.

The chase. A *chase step* of Graph G (converted from documents $\mathcal{D}$) with a set Σ_f of RICOs at a set $\mathcal{F}$ of facts is defined as:

$$\mathcal{F} \Rightarrow_{(\varphi,h)} \mathcal{F}'.$$

Here $\varphi : Q[x_0, y_0](X \rightarrow q_0)$ is a rule in Σ_f and h is a valuation of φ such that the following conditions are satisfied:

(a) All predicates in X are *validated* by $\mathcal{F}$: if p is $x.A \otimes y.B$, then $h(y).B \in [h(x).A]_\otimes$ for $[h(x).A]_\otimes$ in $\mathcal{F}$; similarly for $e_x.A \otimes e_y.B$. If p is $x.A \otimes c$, then $c \in [h(x).A]_\otimes$; similarly for $e_x.A \otimes c$. If p is $\mathcal{M}(x.\bar{A}, y.\bar{B})$, then $\mathcal{M}$ predicts true at $h(x).\bar{A}$ and $h(y).\bar{B}$, and $h(x).A \in [x.A]_{\text{eq}}$, $h(y).B \in [y.B]_{\text{eq}}$ for $A \in \bar{A}$ and $B \in \bar{B}$; similarly for all $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$. If p is $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$, then τ is in $[x.A]_\otimes$ or $[e_x.A]_\otimes$. If p is $T(s, s', \tau)$, then τ is in $[h(x).A]_\otimes$ when s and s' are derived from $h(x).A$ and $h(y).B$, and τ is in $[h(e_x).A]_\otimes$ when s and s' are from $h(e_x).A$ and $h(e_y).B$.

(b) The consequence p_0 extends $\mathcal{F}$ to $\mathcal{F}'$, by adding c, $h(y).B$, $h(e_y).B$ or τ to $[h(x).A]_\otimes$ in $\mathcal{F}'$ if p_0 is $x.A \otimes c$, $x.A \otimes y.B$, $e_x.A \otimes e_y.B$, $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$, respectively, where τ is a specific contradiction type (*e.g.,* causal) in $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$. That is, the chase expands ground truth with validated facts in the process.

A *chasing sequence* ξ of G by (Σ, Γ) is a sequence

$$\mathcal{F}_0, \ldots, \mathcal{F}_n,$$

where $\mathcal{F}_0$ is Γ and for each $i \in [1, n]$, there exist φ in Σ and valuation h of φ such that $\mathcal{F}_{i-1} \Rightarrow_{(\varphi,h)} \mathcal{F}_i$ is a valid chase step, *i.e.,* $\mathcal{F}_i$ is valid.

The sequence ξ *terminates* when no more rules in Σ can be applied to further extend $\mathcal{F}_n$ with new facts. Such a sequence is referred to as a *terminal chasing sequence.* When a sequence terminates, we refer to the set $\mathcal{F}_n$ of facts as *the result of* ξ.

B.2 The Church-Rosser Property of Chase

Termination and consistency guarantees. Our chase procedure conducts deep contradiction detection, as each chase step may trigger the next step with new facts. Moreover, under the assumption that all rules in Σ and initial facts in Γ are correct, and that ML and textual predicates are accurate, no incorrect fact can enter the final result $\mathcal{F}_k$, which can be proved by induction on chase steps.

We further show that the chase process over graph G is Church-Rosser, *i.e.,* the final result of any terminal chasing sequence ξ is unique regardless of the order in which rules are applied.

Proposition 4: *The chase process over graph G with rules Σ is Church-Rosser.* □

Proof: The proof proceeds in two steps.

(1) *All chasing sequences are finite.* Each chase step adds a new fact to a finite set of attributes of vertices and edges. Since the number of nodes and edges in graph G is finite, and each attribute can only be validated once to a consistent set of values or types, the number

of possible valid chase steps is bounded.

Specifically, for any terminal chasing sequence $\xi = (\mathcal{F}_0, \ldots, \mathcal{F}_n)$ of G under (Σ, Γ), we observe that: (a) Each fact assigns a constant c (from Γ or derived from models) to an attribute $x.A$ or $e_x.A$, which can happen at most $O(|\Gamma||G|)$ times. (b) Each fact may equate two attributes (*e.g.,* $x.A = y.B$, or $e_x.A = e_y.B$), which is bounded by $O(|G|^2)$, as the number of such unordered pairs is finite. (c) Each contradiction type label (*e.g.,* $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$) is added at most once per pair of attributes and contradiction type τ, again bounded by $O(|G|^2)$. Thus, the total number of distinct and valid facts is bounded by a polynomial in $|G|$ and ξ is terminate.

(2) *All terminal chasing sequences converge to the same result.* Assume by contradiction that two terminal sequences $\xi_1 = (\mathcal{F}_0, \ldots, \mathcal{F}_{n_1})$ and $\xi_2 = (\mathcal{F}'_0, \ldots, \mathcal{F}'_{n_2})$ over the same graph G and rules Σ produce different fact sets $\mathcal{F}_{n_1} \neq \mathcal{F}'_{n_2}$. Then there exists a chase step $\mathcal{F}'_i \Rightarrow_{(\varphi,h)} \mathcal{F}'_{i+1}$ in ξ_2 such that some new fact (say, $h(x).A \oplus c$ or $h(x).A \oplus h(y).B$ or $\tau(h(x).A, h(y).B)$) appears in $\mathcal{F}'_{i+1}$ but does not appear in $\mathcal{F}_{n_1}$. However, by the validity of rules and consistency of ML and textual classifiers, the rule φ is still applicable to $\mathcal{F}_{n_1}$ via the same valuation h. Therefore, we can extend $\mathcal{F}_{n_1}$ with the same fact, which contradicts the assumption that $\mathcal{F}_{n_1}$ is terminal. Moreover, once the ML and textual models $\mathcal{M}$ and T are trained and validated, they produce consistent outputs across chase sequences. Hence, any rule φ applied in ξ_2 would also apply in ξ_1 under the same data and valuation, leading to convergence.

Thus, all terminal chasing sequences yield the same fact set. □

B.3 Parallel detection algorithm

We next develop Detector, a parallel algorithm for checking contradictions, and shows its parallel scalability [74].

Given a document collection $\mathcal{D}$, a unified graph G, and an RICO set Σ, Detector detects all intra- and inter-document contradictions. It follows the Bulk Synchronous Parallel (BSP) model [115] with one master and K worker processors. The global graph G is partitioned across workers for parallel evaluation. Let G_D denote the subgraph induced by document $D \in \mathcal{D}$. The partitioning strategy satisfies:

- Each document-level graph G_D is kept intact within a single partition to enable efficient local rule execution whenever possible. Large graphs are further decomposed by clustering (dual) star-shaped substructures to preserve logical coherence.
- For inter-document conflict detection (*e.g.,* entity co-reference contradictions), document pairs with potential overlap are assigned to the same partition when possible.

Following [53], each worker node receives a graph fragment $G^{(i)}$ and executes rule checks in Σ locally over its vertices and edges. The master node coordinates execution, uploads tasks via SSH, and aggregates the violation reports returned by workers.

Conflict rules and local evaluation. We discover a RICO rule set Σ capturing diverse contradiction types. Each rule is implemented as a local validator over a graph partition. Each worker evaluates all applicable rules on its assigned fragment independently, without inter-worker communication, enabling fully parallel execution. The results are serialized into a violation report containing offending node IDs, violated rule IDs, and relevant context.

Example 7: Consider the following scenario over three documents:

- Doc1: John Smith is born in 1980 and is the father of Mary.
- Doc2: John Smith is recorded as a **child** of Mary born in 1975.
- Doc3: John Smith is listed as 25 years old in 2025.

The following contradictions are automatically flagged:

- **Temporal contraction**: A cross-check between Doc1 and Doc2 reveals that the birth year (1980) postdates his status as a child in 1975.
- **Numeric contradiction**: The Master node joins Doc1 and Doc3 to identify that the derived age (45) contradicts the recorded 25.
- **Content contradiction**: The fusion of Doc1 and Doc2 triggers a role conflict, as the same person cannot be both *parent* and *child* of Mary.

These violations are detected independently in their respective partitions and collected by the master for final output. □

Complexity and Scalability. Let $|\mathcal{D}|$ be the number of documents and $|\Sigma|$ the number of RICO rules. We analyze the complexity of Detector as follows. (1) *Local evaluation.* For a large document graph G_D, instead of a full scan, Detector employs f_{index} to extract only the subgraphs relevant to the candidate patterns in Σ. Thus, the local cost is $O(\text{avg}(|G_{sub}|) \cdot |\Sigma|)$, where $|G_{sub}| \ll |G_D|$ represents the localized context of a potential contradiction. This ensures that the evaluation remains efficient even as document length grows. (2) *Master coordination.* Input distribution and result aggregation across K workers cost $O(K)$ in the worst case. (3) *Parallel efficiency.* Since checks are rule- and graph-local, the procedure is embarrassingly parallel. With K worker processes, the expected time is

$$T(n, K) = O\left(\frac{|\mathcal{D}|}{K} \cdot |\Sigma| + \log K\right).$$

Thus the runtime scales approximately inversely with K: as the number of processors increases, the cost decreases nearly linearly, demonstrating the parallel scalability in the sense of [74].

B.4 Document marking

Conflict indexing. To enable conflict indexing from contradictions in G_D back to their textual contexts in D, each node and edge is annotated with a `text_span` field during graph conversion (Section 5.1), recording its location in D. This is done using two strategies: (1) *Exact matching:* direct string matching of entity and relation mentions via regular expressions; and (2) *Fuzzy matching:* when exact matches fail, a windowed search using sequence similarity (*e.g.,* Levenshtein ratio [18]) selects the most likely text span.

We define a function $f_{\text{index}}(u) \rightarrow [s_u, e_u]$ that maps each $u \in G_D$ (node or edge) to a text span $[s_u, e_u]$ in the source document, where s_u and e_u represent the start and offset indices in the source document D, respectively. This enables later stages to trace graph components back to their textual origin for contradiction localization and context retrieval.

To map graph-level contradictions back to text: (1) we extract all triples $T = \{(h, r, t)\}$ from the raw text and filter them by the user-specified entity set $\mathcal{E}$, restricting detection to relevant entities; (2) for each retained triple (h, r, t), we apply f_{index} to locate the sentences containing the head and tail entities, providing precise textual anchors; and (3) we localize logical or semantic conflicts

identified by RICOs (Section B.3) within both the triples and their associated text spans. If a statement in D conflicts with multiple claims, all corresponding spans are jointly marked, supporting multi-point tracing and relational analysis.

Example 8: Consider the entity "Scrooge" in a text, from which the following triples are extracted: (Scrooge, is_generous_to, poor) and (Scrooge, is_stingy_to, poor). RICOs similar to those in Example 2 identify a logical contradiction between these two claims. Then, an example of the document annotation using $f_{\text{index}}(u) \rightarrow [s_u, e_u]$ is:

- **Triple 1:** `(Scrooge) -[is generous to]-> (poor)`
  - Head (s_u): "Scrooge has always been generous to the poor during Christmas."
  - Tail (e_u): "The poor often receive gifts from Scrooge."
- **Triple 2:** `(Scrooge) -[is stingy to]-> (poor)`
  - Head (s_u): "Despite rumors, Scrooge is known to be stingy with his wealth."
  - Tail (e_u): "The poor rarely see any charity from Scrooge."

Thus, we successfully map the contradictions detected by the RICOs in the attributed graph G_D back to their corresponding textual contexts (Head and Tail) in the original document D. □

Complexity analysis. The overall complexity of the algorithm is $O\left(\frac{N}{M} \cdot C_{\text{model}} + T \cdot N\right)$, where N is the total number of tokens, M is the chunk size, T is the number of extracted triples, and C_{model} is the cost per model inference. The pipeline proceeds as follows. (1) *Triple extraction:* the text is divided into $\frac{N}{M}$ chunks, each triggering one model call, yielding $O\left(\frac{N}{M} \cdot C_{\text{model}}\right)$. (2) *Entity matching and localization:* based on regex and sentence segmentation, costing $O(N)$. (3) *Fuzzy matching:* at worst $O(T \cdot N)$ across T triples, though windowing and thresholding reduce cost in practice. (4) *Contradiction detection:* scales linearly with the number of chunks, with low overhead. Overall, the method scales well with input size and remains efficient even for long and complex documents.

C LLM PROMPTS

We present LLM prompts used in various stages of DocChecker.

C.1 LLM prompt for graph generation

The prompt for graph generation is as follows:

Graph Generation Prompt for GPT-5.2

(1) Role and Task
You are an expert knowledge graph extractor. Your task is to convert the provided document context into a precise `JSON Property Graph` (G_D). The graph must only focus on the relationships between the specified target entities based on the given context excerpts.

(2) Input Data
- **Target Entities ($\mathcal{E}$)**: A user-specified set of entities to be included as vertices in the graph.
- **Relevant Contexts**: Specific text excerpts from the original document (D) describing relationships.

(3) Input Content (Example)
- **Target Entities**: `Elon Musk, SpaceX, X (Twitter), Hawthorne, CA`
- **Context Excerpts**:
  `(1) Elon Musk, the CEO of SpaceX.`
  `(2) SpaceX (founded in 2002), acquired X in 2022.`
  `(3) SpaceX has its headquarters in Hawthorne, CA.`

Note: Context excerpts are based on the original document text.

(4) Output Specification
Generate the property graph G_D as a single JSON object with `"vertices"` and `"edges"` arrays.
- **Vertices**: Required fields: `id`, `label`, `name`, `attributes`.
- **Edges**: Required fields: `source`, `target`, `label`, `attributes`.

(5) Desired Output Format
- **JSON Structure**:
```
{ "vertices": [
{"id": "v1", "label": "...", "name": "...", "attributes":
{...}}, ...
],
"edges": [
{"source": "vA", "target": "vB", "label": "...",
"attributes": {...}}, ... ] }
```

(6) Expected Output Example
- **Generated JSON**:
```
{ "vertices": [
{"id": "v1", "label": "Person", "name": "Elon Musk",
"attributes": {"Role": "CEO"}},
{"id": "v2", "label": "Company", "name": "SpaceX",
"attributes": {"Founded": 2002}}
],
"edges": [
{"source": "v1", "target": "v2", "label": "IS_CEO_OF",
"attributes": {}} ] }
```

C.2 LLM prompt for pruning search space and generating RICO predicates

The LLM Prompts are as follows:

Contradiction Analyzer and Rule Predicate Generator Prompt

(1) Role and Task
You are a highly specialized Contradiction Analyzer and Rule Predicate Generator. Your task is to review a set of Dual Star Patterns ($\mathbb{Q}$) extracted from a knowledge graph, identify those containing a contradiction, classify the primary contradiction type, and propose relevant `RICO` predicates for rule mining.

(2) Input Format
You will receive dual star patterns in JSON format. Each dual star (Q_i) consists of two star structures linked by a `same_entity` edge, including their multi-hop neighbors and full attributes.

(3) Required Output Format
Return a single JSON list containing only the identified contradictory patterns ($\mathbb{Q}_s$). For each Q_i', include:
- `"contradiction_type"`: One of the 7 types defined below.
- `"confidence_score"`: A float from 0.0 to 1.0.
- `"candidate_predicates"`: A list of 3–5 relevant predicates (*e.g.,* `["HasAttribute(X, Y)", "HasValue(X, V)"]`).

(4) Constraint
The analysis must strictly adhere to the provided definitions. If a pattern is not contradictory or does not clearly fit a type, omit it from the output.

(5) Contradiction Type Definitions
- **1. Factual**: Requires external world knowledge to verify (external

Algorithm 2: Parallelized Algorithm PLearner

Input: Same as Algorithm 1
Output: Same as Algorithm 1

1 Reuse lines 1-11 from Algorithm 1 ;
2 **while** $(f_c(\Sigma, C) \leq \alpha)$ ***and*** (FailCount < 3) **do**
3 $\Sigma_{\text{new}} \leftarrow \emptyset$;
4 **Distribute** the set of potentially inconsistent structures $\mathbb{Q}_s$ across K processing units;
5 **foreach** *structure* Q_i' *in the distributed set* ***processed simultaneously*** **do**
6 The processing unit uses the LLM to generate rule proposals Σ_i ;
7 Filter proposals: keep only rules Σ_i' that correctly identify contradictions in the ground truth C;
8 **Collect** Σ_i' into Σ_{new} (ensure safe aggregation);
9 $\Sigma' \leftarrow \Sigma' \cup \Sigma_{\text{new}}$;
10 Parallelize Generalize predicates in Σ';
11 Use generalized Σ' as few-shot prompt for LLM to summarize patterns and guide next rule generation;
12 **if** $(\text{iter} + 1) \bmod 3 = 0$ **then**
13 Partition corpus $\mathcal{D}$ into M disjoint fragments $\{\mathcal{G}_1, \ldots, \mathcal{G}_M\}$;
14 **Parallel Distribute** fragments to K workers ;
15 **foreach** *fragment* $\mathcal{G}_m$ ***processed in parallel*** **do**
16 Apply current rules Σ' to local subgraph $\mathcal{G}_m$;
17 Compute local statistics (hits, accuracy, coverage) ;
18 **Reduce** local statistics to obtain global $f_a(\mathcal{D}_{\text{clean}})$ and $f_c(\Sigma, C)$;
19 **if** $f_a(\mathcal{D}_{\text{clean}}) > f_a(\mathcal{D})$ **then**
20 $\Sigma \leftarrow \Sigma \cup \Sigma'$;
21 FailCount $\leftarrow 0$;
22 **else**
23 FailCount $\leftarrow$ FailCount $+ 1$;
24 $\Sigma' \leftarrow \emptyset$
25 **return** Σ;

KBs needed).
- **2. Content**: Caused by altering one or more attributes of an event or entity.
- **3. Causal**: Arising from a mismatch between cause and effect.
- **4. Negation**: Resulting from negating original sentences (antonyms or "not").
- **5. Relation**: Mutually exclusive relationships between entities (reversed or mismatched).
- **6. Numeric**: Mismatched numbers or values outside an acceptable range.
- **7. Temporal**: Caused by updates over time affecting temporal order.

D LEARNING RICOS IN PARALLEL

Parallel algorithm PLearner is shown as Algorithm 2. We present its implementation details as follows.

Rule Generation Stage Parallelism. To avoid lock contention while preserving global consistency, PLearner adopts an asynchronous producer-consumer design with a lock-free coordination layer (line 4). Initially, the controller distributes the contradictory patterns across units (line 5). Each worker acts as a producer (line 6), pushing its generated rule set Σ_i' into a thread-safe high-concurrency queue (line 7). A dedicated collector thread consumes these batches and performs atomic, hash-based deduplicated unions to construct Σ_{new} (line 8). PLearner aggregates all discovered Σ_{new} into a unified Σ' upon completion of all worker processes (line 9). This decouples high-latency LLM generation from low-latency aggregation, ensuring consistent global state under high throughput. To balance workload across heterogeneous patterns, a dynamic task-stealing mechanism ensures all K workers complete iterations synchronously.

To ensure autonomous scalability in these steps, PLearner uses a *Token-Bucket* controller to manage traffic. (1) *Smart Re-scheduling.* Instead of a simple exponential backoff, which can waste significant time by forcing workers into long, idle waits, our workers perform a *task swap.* When a rate-limit error occurs, the current task is temporarily sidelined, and the worker immediately switches to other local graph processing tasks that do not need the API. (2) *Dynamic Flow Control.* It monitors how often workers are being blocked and automatically adjusts the speed of requests. This keeps the pipeline moving at the maximum safe speed, avoiding the "stop-and-start" cycles that typically kill performance in large-scale runs.

Verification parallelism. PLearner parallelizes this stage while keeping meta-learning feedback as a synchronized global update. First, predicates in Σ' are generalized in parallel (line 10), and the generalized Σ' is used as few-shot prompts for LLM pattern summarization (line 11). Then, PLearner distributes the verification stage of Learner (lines 12–25), the most costly component, across K workers.

PLearner adopts graph-centric parallel model following GRAPE [43]: (1) *Partitioning phase.* The corpus $\mathcal{D}$ and attributed graph are partitioned into M disjoint fragments $\{\mathcal{G}_1, \ldots, \mathcal{G}_M\}$ (line 13) and dispatched to K workers while preserving local structure (line 14). (2) *Local Match phase.* Each worker independently applies Σ' to its fragment (lines 15–17), leveraging dual-star symmetry to detect contradiction instances locally, without cross-worker communication. (3) *Global Consolidation.* Partial statistics are aggregated via a single-step global reduction (line 18) to compute f_a and f_c. If accuracy improves, rules are committed to Σ (line 20).

By decoupling fragment-level contradiction matching from global metric computation, this design avoids the high synchronization overhead typically associated with iterative graph processing, yielding near-linear scalability with respect to corpus size.

Proposition 5: PLearner *scales parallelly relative to* Learner. □

Proof: Let T_{seq} and T_{par} denote the execution time of the sequential Learner and the parallel PLearner with K workers, respectively. We show that PLearner achieves near-linear speedup as the number of candidate patterns $|\mathbb{Q}_s|$ increases, *i.e.,* its parallel scalability.

Sequential cost. The cost of Learner is dominated by rule generation:

$$T_{\text{seq}} \approx \sum_{i=1}^{|\mathbb{Q}_s|} t_{\text{llm}}(i) + T_{\text{other}},$$

where $t_{\text{llm}}(i)$ is the latency of the LLM call for pattern Q_i', and T_{other} is the remaining overhead (e.g., pruning and aggregation).

Parallel decomposition. PLearner evenly distributes the $|\mathbb{Q}_s|$ patterns across K workers. The parallel rule generation cost is

Model	SNLI				MNLI			
	Acc	P	R	F1	Acc	P	R	F1
DocChecker	0.870	0.879	0.989	0.931	0.770	0.899	0.844	0.870
bert-base-uncased	0.671	0.675	0.671	0.671	0.600	0.603	0.600	0.599
roberta-base	0.698	0.706	0.698	0.698	0.716	0.716	0.716	0.715
xlnet-base-cased	0.709	0.710	0.709	0.709	0.694	0.693	0.694	0.693

Table 7: Performance on sentence-Level NLI tasks

$$T_{\text{gen_par}} = \max_{j\in[1,K]} \sum_{i\in \text{Worker}_j} t_{\text{llm}}(i).$$

Assuming roughly uniform LLM latency and $|\mathbb{Q}_s| \gg K$, the workload is balanced, we have the following:

$$T_{\text{gen_par}} \approx \frac{1}{K} \sum_{i=1}^{|\mathbb{Q}_s|} t_{\text{llm}}(i).$$

Overhead. Predicate generalization is embarrassingly parallel and incurs no inter-worker communication. Aggregating Σ'_i into Σ_{new} adds only $O(\log K)$ synchronization, negligible compared with t_{llm}.

Scalability. As $|\mathbb{Q}_s|$ grows, $T_{\text{other}}/T_{\text{seq}} \to 0$, and thus

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}} \approx \frac{T_{\text{seq}}}{T_{\text{seq}}/K} = K.$$

Hence, PLearner is parallelly scalable relative to Learner, assuming LLM rate limits are not the bottleneck. □

E CONSTRUCTING THE GOLD STANDARD C

To evaluate rule correctness against human annotations, we define a transformation $\Phi : \mathcal{D} \to C$ that maps textual ground truth into structured verification instances. For each document $d \in \mathcal{D}$, the dataset provides $\langle T_d, E_d, L_{\text{type}}, P_{\text{plug}}\rangle$, where T_d is the full text, E_d is the evidence span, L_{type} is the contradiction category, and P_{plug} records the perturbation (*e.g.,* `Replace`). The construction of a gold instance $c \in C$ proceeds in three steps.

(1) Entity–attribute extraction. Following Section 5.1, we extract entities $\mathcal{E}$ and attributes $\mathcal{A}$ appearing in E_d. Each fact $f = (s, p, o)$ anchors s and o to nodes in the enriched graph G^+.

(2) Predicate mapping. The annotated contradiction label L_{type} is mapped to a typed predicate $P \in \mathsf{P}$; for example, a "Causal" label maps to a causal dependency predicate.

(3) Contradiction modeling. The set P_{plug} determines inconsistency boundaries. For example, if $P_{\text{plug}} =$ `Replace`, any value $v' \neq v_{\text{orig}}$ on the same attribute path constitutes a contradiction.

Example 9: Consider ContraDoc instance `3488771835_6`, where E_d = "One rat, Jenner, agreed wholeheartedly with The Plan," annotated as *Perspective/Opinion* and *Emotion/Mood/Feeling* with perturbation `Replace`. Algorithm PLearner extracts subject *Jenner*, object *The Plan*, and a positive sentiment attribute, yielding c_1 = {Subj: Jenner, Pred: Has Emotion, Obj: The Plan, Val: Positive}. As `Replace` enforces semantic inversion, C encodes the constraint that any rule detecting a negative/conflicting sentiment on this entity pair is a correct detection. Thus, C provides a fixed structured reference for computing f_a and f_c (Alg. 1, lines 9–25). □

F ADDITIONAL EXPERIMENTAL DETAILS

F.1 Results of Additional Experiments

We also conduct the following experiments.

Sentence-level NLI. This task tests premise–hypothesis classification, a benchmark for fine-grained semantic understanding. As shown in Table 7, DocChecker performs best, reaching 87.0% Acc ($F_1 = 0.931$) on SNLI and 77.0% Acc ($F_1 = 0.870$) on MNLI. While `roberta-base` and `xlnet-base-cased` are competitive on MNLI (71.6% and 69.4% Acc), they still lag behind DocChecker. The 17.2% Acc margin over RoBERTa on SNLI further confirms that DocChecker effectively captures core semantic relations.

F.2 Details of Experimental Datasets

We provide details of the experimental datasets below.

Domain of datasets. We categorize the 15 datasets into three groups based on structural complexity and domain characteristics:

(1) Document-level & real-world contradiction. ContraDoc targets cross-document conflicts; ECON evaluates consistency in event-centric narratives; Knowledge Conflict captures discrepancies between model knowledge and external evidence; Real-life Contradiction Corpus contains naturally occurring contradictions from news and social media; and ConTRoL benchmarks reasoning-intensive contradiction detection.

(2) Specialized & domain-specific contradiction. ContractNLI and CaseHOLD focus on legal consistency and reasoning; StorySumm evaluates inconsistencies between stories and their summaries; and DOCNLI addresses large-scale document-level inference.

(3) Standard NLI & linguistic probing. SNLI/MNLI benchmark general domain inference; RTE-3 is a classic entailment challenge set; MCTest-NLI derives contradiction cases from machine comprehension; binary-FEVER-NLI targets fact-verification conflicts; and Negation probing sets isolate logical negation phenomena.

Statistical Summary and Contradiction Distribution. We summarize dataset scale, ground truth labeling, and contradiction-type distribution across all 15 datasets. For QA datasets, ground truth derives from correct answers, with alternative candidates treated as contextual contradictions. For entailment/non-entailment datasets, we use LLM annotation with expert verification to identify contradiction types within non-entailment instances; entailment and neutral cases are treated as non-contradictory negatives.

The **binary-FEVER-NLI** dataset [130] is a large-scale fact-checking benchmark adapted from FEVER, containing 221,361 samples (203,152 train/8,209 dev/10,000 test). Labels are binary (*Entailment* vs. *Not-Entailment*), with Not-Entailment (41.28%) merging original Contradiction and Neutral categories to evaluate non-supportive evidence detection. As a Wikipedia-based verification task, its contradiction profile is dominated by *Factual* and *Relational* conflicts, with moderate *Content*, *Temporal*, and *Numeric* mismatches, and relatively fewer *Causal* and *Negation* cases.

The **CaseHOLD** dataset [136] is a large-scale legal benchmark with 53,000 multiple-choice problems derived from U.S. federal and state court opinions. Each question requires selecting the correct legal holding from five options (one ground truth and four expert-

crafted distractors). Unlike standard NLI, the "not-entailment" candidates are often semantically plausible yet legally inconsistent, making CaseHOLD a demanding test of domain-specific legal reasoning. Our categorical analysis shows that *Factual* and *Causal* conflicts dominate, reflecting the role of complex precedent and legal dependency structures, while *Content* and *Temporal* contradictions also appear frequently. In contrast, *Numeric*, *Relational*, and *Negation* contradictions are relatively rare, as the primary challenge lies in resolving high-level judicial logic rather than surface-level linguistic or numeric inconsistencies.

ConTRoL [83] is a specialized benchmark for contrastive reasoning over long documents, consisting of 8,325 expert-annotated samples derived from Wikipedia. Formulated primarily as a multiple-choice task, it requires models to cross-reference dispersed thematic segments within the same document to resolve conflicts. The ground truth is determined through logical synthesis across these disjoint evidence chains, making ConTRoL a rigorous test of long-range reasoning and cross-paragraph coherence. Our categorical analysis shows that *Casual* and *Content* contradictions dominate (approximately 65% combined), reflecting the emphasis on semantic consistency and topic alignment. These are followed by notable *Factual* and *Temporal* ones, which arise from precise cross-block comparisons. In contrast, simple *Negation* and *Numeric* inconsistencies are less prevalent, as the dataset prioritizes higher-level causal and logical integration over surface-level linguistic cues.

ContraDoc [79] is a high-quality benchmark for evaluating self-contradiction detection in long-form documents (300–2,200 tokens). It contains 891 documents across three domains (Wikipedia, News, and Story Summaries), with binary ground truth: 449 contradictory (`pos`) and 442 non-contradictory (`neg`) samples. To assess contextual difficulty, contradictions are categorized by span: Intra-sentence (73 documents), Local (within 4 sentences; 220 documents), and Global (beyond 4 sentences; 155 documents). Our analysis indicates that *Numeric* and *Negation* contradictions are most frequent and relatively easier to detect due to their objective nature. These are followed by *Factual*, *Content*, *Casual* and *Temporal* inconsistencies, which typically appear in the Global scope. In contrast, *Relational* contradictions are less common and harder to identify, as they require deeper logical synthesis over extended context.

ContractNLI [70] is a specialized legal benchmark comprising 607 real-world non-disclosure agreements (NDAs) and 10,319 hypothesis-premise pairs. It adopts a three-way ground truth: *Contradiction* (0), *Entailment* (1), and *Neutral* (2) and poses greater difficulty than general NLI due to complex legal syntax and strict logical constraints. Our analysis shows that *Content* and *Casual* conflicts dominate, typically arising from prohibited actions or conditional exclusions within clauses. These are followed by *Factual* and *Temporal* inconsistencies, often involving discrepancies in confidentiality duration or scope. *Relational* and *Numeric* contradictions appear at moderate levels (*e.g.*, damages or notice periods), while *Negation* conflicts are rare, reflecting the prescriptive nature of contractual language rather than descriptive narrative logic.

DOCNLI [130] is a large-scale document-level NLI benchmark of 942,000 samples, constructed by reformulating five heterogeneous NLP tasks (including ANLI and SQuAD) into inference settings. It pairs full-document premises with hypotheses of varying granularity to evaluate long-range reasoning. The ground truth is provided in both binary (*Entailment* vs. *Not-Entailment*) and three-way formulations. We find that *Causal*, *Content* and *Factual* contradictions dominate, reflecting its summarization-oriented construction. *Relational* and *Temporal* inconsistencies also appear frequently and typically require cross-paragraph evidence alignment. In contrast, *Numeric* and *Negation* conflicts are comparatively rare, representing the most challenging logical categories within the corpus.

ECON [66] is a specialized benchmark for evaluating event-centric consistency in long-form narratives. It contains approximately 12,000 samples assessing whether a generated story remains logically coherent with its established event history. The ground truth is binary, distinguishing between *consistent* (entailment) and *inconsistent* (contradiction) event flows. Our categorical analysis of the contradiction types shows that *Factual* and *Numeric* conflicts are most dominant, reflecting the dataset's core focus on the logical "effects" of events and their chronological ordering within a narrative. *Casual*, *Relational* and *Temporal* inconsistencies appear next in frequency, often arising when the logical consequences of a clause are violated or when the duration of legal obligations (such as confidentiality periods) deviates from the agreed-upon timeframe. In contrast, *Negation* and *Content* contradictions are relatively rare, as the challenge centers on maintaining high-level narrative coherence rather than detecting surface-level discrepancies.

Knowledge Conflict [121] is a diagnostic benchmark designed to evaluate how LLMs resolve discrepancies between internal parametric knowledge and external contextual evidence. It contains over 2,000 samples spanning diverse domains and entities, constructed via two synthetic mechanisms: *substitution* (in-domain entity replacement) and *shuffle* (entity reordering). The ground truth is multi-faceted, requiring models to (i) detect the existence of conflicts, (ii) localize the contradictory segments, and (iii) articulate divergent viewpoints generated from parametric versus contextual knowledge. Our categorical analysis of the seven contradiction types indicates that *Factual* and *Temporal* conflicts dominate, reflecting the dataset's emphasis on manipulating entity properties and the specific time-sensitive states of entities via substitution. *Content* and *Numeric* discrepancies appear with moderate frequency due to the knowledge-intensive nature of the corpus. By contrast, *Negation*, *Causal*, and *Relational* contradictions are relatively rare, since the generation paradigm primarily targets semantic entity rewrites rather than logical or chronological inversions.

The **binary-MCTest-NLI** dataset [130] is a narrative-driven benchmark adapted from the MCTest machine reading comprehension dataset, containing 8,000 samples split into 4,800 for training, 2,400 for testing, and 800 for development. The task follows a binary classification scheme (*Entailment* vs. *Non-entailment*), with a fixed label distribution of 75% Non-entailment (6,000 samples) and 25% Entailment (2,000 samples). Our categorical analysis of the seven contradiction types within the Non-entailment class indicates that *Content*, *Factual* and *Casual* discrepancies are most prevalent, reflecting the story-centric nature of the corpus where inconsistencies commonly arise from character actions and entity attributes. *Numeric* and *Temporal* conflicts follow in frequency, of-

ten stemming from narrative event tracking. By contrast, *Negation*, and *Relational* contradictions are less frequent, as the primary challenge emphasizes lexical coherence and story-level reasoning rather than explicit logical or complex entity-to-entity interdependencies.

MNLI (Multi-Genre Natural Language Inference) [123] is a large-scale, multi-domain benchmark containing 412,313 samples across diverse text genres. The ground truth adopts a three-way classification (*Entailment*, *Neutral*, *Contradiction*), with Contradiction instances constituting approximately 32.53% (134,116 samples). Our analysis shows that *Casual* and *Factual* conflicts are most prevalent, reflecting the frequent occurrence of premise-driven effects and entity-based contradictions across genres. These are followed by *Negation* and *Content* ones, which highlight MNLI's emphasis on linguistic diversity and direct semantic opposition. In contrast, *Numeric*, *Temporal*, and *Relational* contradictions are present but less dominant, as the dataset primarily targets broad natural language understanding rather than specialized mathematical logic, chronological reasoning, or complex entity linking.

Negation (RTE-Negation) [23] is a specialized diagnostic corpus constructed by perturbing the RTE2 benchmark with explicit negative markers. It contains 899 samples partitioned into a development set (102 pairs) and a test set (797 pairs), with a binary ground truth (*YES*/*NO*) and a nearly balanced label distribution (50.17% contradictions; 451 samples). We find that *Factual* dominates (>80%), reflecting the dataset's design focus on testing explicit negative facts. *Numeric* and *Negation* ones appear as relatively simple contradiction cases, while *Content* and *Relational* conflicts occur moderately through world-knowledge contrasts. Conversely, *Causal* and *Temporal* types are rare, as the dataset targets direct semantic inversion, not complex event-flow reasoning.

Real-life Contradiction Corpus [23] is a high-quality benchmark of 131 naturally occurring contradiction pairs collected from Google News, Wikipedia, Lexis Nexis, and the DARPA GALE Program. Unlike synthetic datasets, all samples are labeled as contradictions (100% positive), providing a focused evaluation of real-world inconsistency phenomena. Our seven-type categorical analysis reveals a diverse distribution: *Numeric* (∼29.0%) and *Factual* (∼30.5%) dominate, followed by *Casual* (17.56%). *Content* and *Relational* conflicts account for ∼20% collectively, while *Temporal* and *Negation* contradictions form the minority, reflecting the nuanced and context-driven nature of naturally occurring inconsistencies in professional and encyclopedic writing.

RTE-3 [51] represents the transition to three-way textual entailment classification, comprising 2,974 sentence pairs in the 3w-4C configuration. The ground truth is labeled as *Entailment* (1,520), *Contradiction* (340), and *Unknown* (1,114), with contradictions accounting for 11.43% of the corpus. The dataset spans four task domains: Information Extraction (IE), Summarization (SUM), Information Retrieval (IR), and Question Answering (QA)—to evaluate complex logical and commonsense reasoning. We find that *Factual* and *Content* types dominate (∼55%), reflecting entity-role mismatches and semantic overlap common in IE and SUM. *Relational* and *Casual* conflicts follow, particularly in IR and QA, where direct evidence verification is required. *Numeric* and *Temporal* mismatches are less frequent, while *Negation* contradictions are rare, reflecting a focus on structural alignment over explicit logical inversion.

SNLI (Stanford Natural Language Inference) [14] is a foundational large-scale NLI benchmark comprising 570,152 sentence pairs derived from image captions. It adopts a three-way classification scheme: *Entailment* (0), *Neutral* (1), and *Contradiction* (2), with contradictions accounting for 33.27% (189,702 samples), maintaining a near-balanced distribution across training, validation, and test sets. Our seven-type categorical analysis shows that *Content* and *Relational* discrepancies are most dominant, often involving conflicting actions or entity attributes (*e.g.,* "a man sitting" vs. "a man standing"). *Negation* and *Factual* inconsistencies are also frequent, reflecting the crowdsourced hypothesis generation process. *Numeric* and *Temporal* conflicts appear with moderate frequency, typically regarding counts of entities or event timing, while *Causal* contradictions are rare due to the descriptive, caption-based nature of the corpus, which seldom encodes explicit cause–effect relations.

StorySumm [110] is a narrative-centric benchmark for long-form fiction consistency, comprising 32 short stories and 96 primary entries, with 580 associated summary paragraphs. The dataset focuses on narrative reliability, with a high ground-truth error rate of 85.17% (494 paragraphs) labeled as contradictory. Samples are categorized by difficulty, and 41.67% are marked as "hard," requiring deep comprehension of character development and narrative arcs. Our seven-type categorical analysis indicates that *Temporal*, *Causal* and *Factual* conflicts are significantly more frequent than in other datasets, reflecting the reliance of storytelling on chronological progression and character motivation. *Content* and *Negation* discrepancies are also prominent, often involving misattributed roles or plot details. By contrast, *Numeric* and *Relational* errors are relatively rare, as story narratives prioritize plot progression and character dialogue over precise quantification or entity linking..